Chapter 12

Guide to Designing and Implementing an Embedded System Project

12.1 Roadmap

12.1.1 What You Will Learn

After you have studied the material in this chapter, you will be able to:

- Describe how an embedded system project can be developed following an ordered process.
- Design and implement a prototype of an embedded system, including its hardware and software.
- Summarize the fundamentals of the concepts of verification and validation.
- Develop the final documentation of an embedded system.

12.1.2 Review of Previous Chapters

Throughout this book, a smart home system provided with a broad variety of functionalities has been implemented. The NUCLEO board was used as the system core, and many hardware modules and elements were connected to it. A learn-by-doing approach was used, by means of which several embedded system programming concepts were introduced.

The smart home system project was started from scratch in Chapter 1, and functionality was gradually added through the chapters as different hardware modules and elements were incorporated. This approach led to a single file having hundreds of lines, after which the idea of software modularization was introduced in Chapter 5. From then on, in Chapters 6 to 11, many software modules were included as more hardware was incorporated into the system.

The reader may have noticed that, for pedagogical reasons, the features and functionalities of the smart home system were not established at the beginning. This made it possible to introduce the topics gradually, but also led to many changes during its implementation. As a consequence, it can be concluded that it would be more convenient to adopt a structured process to efficiently design and implement an embedded system. This process should include a step at the beginning where the features and functionality of the system are defined.

12.1.3 Contents of This Chapter

In this chapter, a structured process will be introduced to efficiently design and implement an embedded system. The proposed process consists of ten steps, including the selection of the project, its definition, design, implementation, and final documentation. The hardware and software aspects are tackled, following an approach that guarantees consistency between the initial objectives and the obtained results.

In order to illustrate how each of the proposed steps is carried out, a project is implemented within this chapter. For the sake of brevity, a system with a reduced number of sensors and actuators is used in the examples, although this does not limit the introduction of the important concepts.

The proposed process benefits from all the concepts that were introduced throughout this book, while also helping to introduce other important concepts such as *requirements*, *verification*, and *validation*. This chapter constitutes a summary of the book, while introducing important new concepts.

12.2 Fundamentals of Embedded System Design and Implementation

12.2.1 Proposed Steps to Design and Implement an Embedded System Project

The proposed process to design and implement embedded systems is summarized in Table 12.1. It can be seen that it consists of ten steps, ranging from the selection of the project that will be implemented to its design, implementation, and final documentation, as was mentioned in the previous section.

Table 12.1 Summary of the proposed steps to design and implement an embedded system project.

Step	Outcome
1. Select the project that will be implemented	A rationale that leads to an appropriate project to implement
2. Elicit project requirements and use cases	A concise and structured description of what will be implemented
3. Design the hardware	Diagram of hardware modules, connections, and bill of materials
4. Design the software	Diagram of the software design and description of the modules
5. Implement the user interface	Software implementation
6. Implement the reading of the sensors	Software implementation
7. Implement the driving of the actuators	Software implementation
8. Implement the system behavior	Software implementation
9. Check the system behavior	Assessment of accomplishment of requirements and use cases
10. Develop the final documentation	A reference to the most relevant documentation of the project

The proposed steps include a gradual implementation of the software. First, the user interface is implemented in step 5. The aim is to have a way to read the system information and to enter commands. In this step, the reading of the sensors is replaced by *stub* code that temporarily substitutes the *yet-to-be-developed* code to read the sensors. In step 6, the reading of the sensors is implemented and the corresponding values shown on the user interface. In step 7, the actuators are driven, but stub code is used to trigger their activation. In step 8, the complete system behavior is implemented, so no more stub code remains in the software.

In the examples below, the ten proposed steps are introduced by means of a given project that is developed from start to end, following a *top-down* approach. This starts by formulating an overview design of the system, and goes on to gradually define and implement each part in detail.

TIP: Through the examples, the reader is encouraged to think of their own project and to develop that project as each step is introduced, by adapting each example to their own requirements. It is recommended that the reader choose a simple project to avoid complications that will distract from the main aim of this chapter, which is to understand and adopt the proposed steps.

NOTE: There are many other possible approaches to tackling the design and implementation of an embedded system, depending on the characteristics of the project, as well as on the size and skills of the developing team. The proposed steps correspond to the implementation of an embedded system prototype by a single developer or a very reduced team. A more complex project may suggest an iterative approach, where the outcomes of the steps are revised and improved more than once.

Example 12.1: Select the Project that will be Implemented

Objective

Introduce the idea that the project to be implemented should result from a decision process.

Summary of the Expected Outcome

As a result of this step, a rationale about the most appropriate project to be implemented should be obtained.

NOTE: This step is particularly important because the results can lead to either a valuable project or a questionable project (in terms of learning, benefits, etc.).

Discussion on How to Implement this Step

In order to select the project, it should first be established which aspects will be analyzed in the decision process. Those aspects will vary depending on the developer profile and skills, the organization where the developer is studying or working, and many other aspects. However, a table summarizing the aspects to be analyzed, as well as the score of each proposed project in each aspect, seems to be a reasonable way to decide which project to implement.

Implementation of this Proposed Step

First, the aspects that will be considered in the decision process should be determined. For the sake of brevity, just a few aspects will be considered in this example. However, an important idea is introduced: different aspects may have different weights in the decision process. To factor all these ideas into the decision process, a quantitative approach will be followed.

Some aspects that could be considered in the decision process are as follows:

- Availability of the hardware
- Utility of the project
- Implementation time

In the particular context of this book, the hardware availability might be considered an important aspect, given that it is convenient for the reader to reuse the hardware from previous chapters. The usefulness of the project could be considered less relevant, because in this context the reader is more concerned about learning than using the project in a real-life application. Finally, the implementation time can be considered an important aspect, given that the aim is to choose a simple project that can be completed in a single chapter.

Some possible projects that could be analyzed include the following:

- A mobile robot that avoids obstacles
- A flying drone fitted with a camera
- A home irrigation system for indoor plants

In this way, Table 12.2 can be obtained, which includes the possible projects and all the proposed aspects with given weights, as per the above discussion. A weighted score for each project is obtained.

The mobile robot that avoids obstacles will require hardware that was not used in this book (wheels, structure, motors, obstacle sensors, etc.), and for that reason was awarded a score of three out of ten points in the hardware availability aspect. The utility of this project is questionable, and for that reason this project again obtains three out of ten points in the corresponding aspect. Finally, this project will take a reasonable implementation time and, therefore, obtains five points in that aspect. Consequently, considering that the aspects are weighted by a factor of ten, five, and eight, respectively, the weighted scores are obtained (30, 15, and 40), and the sum of weighted scores is 85, as can be seen in the last column of Table 12.2.

The flying drone fitted with a camera will require even more specific and complex hardware than the mobile robot, and for that reason it gets two points in the hardware availability aspect. The utility of this project might be considered higher than the utility of the mobile robot, and therefore it gets five points in that aspect. Finally, this project will demand a considerable implementation time and, therefore, this project scores two points in that aspect. As a result, this project gets a sum of weighted scores of 61.

NOTE: The implementation time aspect gets a lower score when the time demand is bigger.

A typical home irrigation system allows the user to set how often and for how long the plants are irrigated. It can be implemented by means of a few buttons, an LCD display, a moisture sensor, and an on/off electro-valve, as will be discussed in Example 12.3. Most of this hardware is already available to the reader or is easy to obtain and use. Therefore, this project gets eight points in the hardware availability aspect. This project can be useful if the reader has some indoor plants, and therefore it gets seven points in the utility of the project aspect. Finally, this project can be implemented with limited effort, and, therefore, it gets eight points in the implementation time aspect. As a result, this project gets a sum of weighted scores of 179.

The home irrigation system for indoor plants project is chosen, as it gets the highest value in the sum of weighted scores, as can be seen in the last column of Table 12.2.

Table 12.2 Selection of the project to be implemented.

Project		Hardware availability (weight: 10)	Utility of the project (weight: 5)	Implementation time (weight: 8)	Sum of weighted scores
Mobile robot that avoids obstacles	Score on each aspect:	3	3	5	–
	Weighted score:	30	15	40	85
Flying drone provided with a camera	Score on each aspect:	2	5	2	–
	Weighted score:	20	25	16	61
Home irrigation system for indoor plants	Score on each aspect:	8	7	8	–
	Weighted score:	80	35	64	179

NOTE: The score in each aspect, as well as the weight of each aspect, is an approximate value only. It should be noted that, in general, a small variation in an aspect score for a given project, or in an aspect weighting, does not modify which project obtains the highest sum of weighted scores shown in Table 12.2.

Proposed Exercise

1. How can the reader use the concepts that were introduced in this chapter to select a project to implement?

Answer to the Exercise

1. The reader should first establish a list of aspects that they would like to consider in the selection of a project. Then, the weighting of each aspect should be determined, and finally the different projects should be scored.

TIP: Consider including aspects such as "How fun the project is," "What will be learned," or "Profitability" as a way to guarantee that the selection reflects the reader's personal motivations.

Example 12.2: Elicit Project Requirements and Use Cases

Objective

Introduce the concepts of *requirements* and *use cases*.

Summary of the Expected Outcome

As a result of this step, the project to be implemented should be clearly defined. This will be done by means of a list of requirements and use cases.

Discussion of How to Implement this Step

This step is based on the concepts of requirements and use cases.

DEFINITION: In product development, a *requirement* is a singular documented physical, functional, or non-functional need that a particular design, product, or process aims to satisfy.

DEFINITION: In software and systems engineering, a *use case* is a list of actions or event steps typically defining the interactions between an actor (for example, a user) and a system to achieve a goal.

The requirements are the basis on which a project is to be developed. Therefore, there are many important criteria that a developer should apply when writing the requirements. Some of these criteria are frequently summarized using the "SMART" mnemonic acronym, as shown in Table 12.3.

Table 12.3 Summary of the SMART mnemonic acronym.

Letter	Term adopted in this book	Meaning
S	Specific	Clearly defined
M	Measurable	Able to be measured
A	Achievable	Able to be achieved
R	Relevant	Targets a significant need
T	Time-bound	Has a time limit or deadline

NOTE: SMART is sometimes used to refer to other criteria. For example, the letter "A" is frequently related to the term "agreed," meaning that the requirements must be agreed with the client.

WARNING: In professional *project management*, every project should have a due date. Therefore, the time-bound criterion applies to the whole project and can also be applied to each requirement. In this example, it is established that the whole project (all the requirements) should be completed in one week.

A use case can be defined in multiple ways. In the scope of this book, the elements shown in Table 12.4 will be used. The reader should note that alongside each is a simplified approach that is appropriate for many projects.

Table 12.4 Elements that will be used to define a use case.

Use case element	Meaning
ID	A unique number that represents a use case
Title	The title that is associated with the use case
Trigger	What event triggers this use case
Precondition	What must be met before this use case can start
Basic flow	The events that occur when there are no errors or exceptions
Alternate flows	The most significant alternatives and exceptions

Lastly, in most cases there are already products on the market that implement the same functionality as the embedded system project that will be designed. Many of those products may be very successful. Therefore, it is recommended to analyze and summarize those products before writing the requirements and use cases of a new embedded system project, as per the example below.

Implementation of this Proposed Step

First, some examples of home irrigation systems are analyzed, as was suggested in the previous paragraph. Table 12.5 summarizes the main characteristics of two products. Based on those characteristics, the requirements indicated in Table 12.6 are established with consideration of what can be achieved.

NOTE: The requirements in Table 12.6 are preliminary requirements that may be modified as the project evolves. If requirements are modified, then the modifications should be clearly indicated in order to avoid misunderstandings.

Table 12.5 Summary of the main characteristics of two home irrigation systems currently available on the market.

Characteristics	Product A	Product B
Water-in port	½-inch connector	½-inch connector
Irrigation circuits	Controls two independent irrigation circuits	Controls one irrigation circuit
Operation modes	Continuous: water flow is controlled by a button Programmed irrigation: irrigation is time-controlled	Continuous: water flow is controlled by a button Programmed irrigation: irrigation is time-controlled
Configuration	The parameters "how often" to irrigate and "how long" to irrigate can be configured for each circuit	The parameters "how often" to irrigate and "how long" to irrigate can be configured
User interface	A rotary control key, two buttons, and a display	A set of buttons and a set of LEDs
Power supply	Two AA batteries	Four AA batteries
Sensors	None	None
Price	80 USD	60 USD

Table 12.6 Initial requirements defined for the home irrigation system.

Req. Group	Req. ID	Description
1. Water	1.1	The system will have one water-in port based on a ½-inch connector
	1.2	The system will control one irrigation circuit by means of a solenoid valve
2. Modes	2.1	The system will have a continuous mode in which a button will enable the water flow
	2.2	The system will have a programmed irrigation mode based on a set of configurations:
	2.2.1	Irrigation will be enabled only if moisture is below the "Minimum moisture level" value
	2.2.2	Irrigation will be enabled every H hours with H being the "How often" configuration
	2.2.3	Irrigation will be enabled for S seconds, with S being the "How long" configuration
	2.2.4	Irrigation will be skipped if "How long" is configured to 0 (zero)
3. Configuration	3.1	The system configuration will be done by means of a set of buttons:
	3.1.1	The "Mode" button will change between "Programmed irrigation" and "Continuous irrigation"
	3.1.2	The "How often" button will increase the time between irrigations in programmed mode by one hour
	3.1.3	The "How long" button will increase the irrigation time in programmed mode by ten seconds
	3.1.4	The "Moisture" button will increase the "Minimum moisture level" configuration by 5%
	3.1.5	The maximum values will be: "How often": 24 h; "How long": 90 s; "Moisture": 95%
	3.1.6	"How long" and "Moisture" will be set to 0 (zero) if the maximum is reached and the button is pressed
	3.1.7	"How often" will be set to 1 if the maximum is reached and the button is pressed
4. Display	4.1	The system will have an LCD display:
	4.1.1	The LCD display will show the current operation mode: Continuous or Programmed
	4.1.2	The LCD display will show the values of "How often", "How long", and "Minimum moisture level"
5. Sensor	5.1	The system will measure soil moisture at one point with an accuracy better than 5%
6. Power supply	6.1	The system will be powered using two AA batteries
7. Due date	7.1	The system will be finished one week after starting (this includes buying the parts)
8. Cost	8.1	The components for the prototype should cost less than 60 USD
9. Documents	9.1	The prototype should be accompanied by a list of parts, a connection diagram, the code repository, and a table indicating the accomplishment of requirements and use cases

Finally, three use cases are defined, as can be seen in Table 12.7, Table 12.8, and Table 12.9.

Table 12.7 Use Case #1 – Title: The user wants to irrigate plants immediately for a couple of minutes.

Use case element	Definition
Trigger	The user realizes that the plants need irrigation immediately
Precondition	The system must be powered on, and the water supply should be connected. The system is not irrigating the plants.
Basic flow	The user presses the "Mode" button to set "Continuous" irrigation. Water starts to flow to the plants. After a couple of minutes, the user presses the "Mode" button to set "Programmed" irrigation. Water irrigation stops.
Alternate flows	1.a. There is no water supply. The user presses the "Mode" button to set "Continuous" irrigation. The solenoid valve is activated, but the plants are not irrigated. 1.b. The user presses the "Mode" button. The user notes that the water is overflowing the plant pot. The user presses the "Mode" button and irrigation stops.

Table 12.8 Use Case #2 – Title: The user wants to program irrigation to take place for ten seconds every six hours.

Use case element	Definition
Trigger	The user wants to establish an irrigation program
Precondition	The system must be powered on, and the water supply should be connected. The system is not irrigating the plants.
Basic flow	The user presses the "How often" button until the value "6 hours" is shown on the display. The user presses the "How long" button until the value "10 seconds" is shown on the display. Irrigation will start in six hours and will last for ten seconds if the measured moisture is below the "Minimum moisture level" configured.
Alternate flows	2.a. There is no water supply. The solenoid valve will be activated, but the plants will not be irrigated.

Table 12.9 Use Case #3 – Title: The user wants the plants not to be irrigated.

Use case element	Definition
Trigger	The user wants the plants to not be irrigated
Precondition	The system must be powered on, and water supplied should be connected. The system is not irrigating the plants.
Basic flow	The user presses the "How long" button until "How long" is set to 0 (zero). Irrigation is skipped, and a legend indicating "Programmed-Skip" is shown.
Alternate flows	3.a. Plants will not be irrigated even if the measured moisture is below the "Minimum moisture level" configured.

Proposed Exercise

1. Define the requirements and three use cases for the project that was selected in the "Proposed Exercise" of Example 12.1.

Answer to the Exercise

1. It is strongly recommended to start by analyzing and summarizing some products that are available on the market. Based on the characteristics of those products and the reader's own ideas, the initial requirements may be established following the format shown in Table 12.6. The use cases can be defined following the format shown in Table 12.7.

TIP: When defining the requirements, keep in mind the SMART criteria discussed above. In particular, consider the time available to implement the project and the relevancy and achievability of each requirement. Moreover, try to clearly define each requirement, and use measurable quantities if possible.

Example 12.3: Design the Hardware

Objective

Design hardware that fulfills the requirements and can be used to implement the software functionality.

Summary of the Expected Outcome

The outcomes of this step are expected to include:

- a diagram of the hardware modules showing all of their interconnections,
- a rationale that argues the most appropriate components with which to implement the hardware modules,
- a connection diagram of the selected components and tables summarizing their interconnections, and
- a bill of materials that includes all the necessary elements to implement the prototype.

Discussion of How to Implement this Step

A reasonable approach for this step might be to start by analyzing designs made by other developers to implement similar requirements; this also includes the previous chapters of this book. A diagram following the ideas shown in Chapter 1 will be the starting point. After this diagram is created, the parts can be selected and their interconnections defined. The final step will be to create the bill of materials.

Implementation of this Proposed Step

Figure 12.1 shows a first proposal for the hardware modules of the irrigation system prototype. It is based on the elements introduced in previous chapters, as well as the assumption that it will be possible to find an appropriate moisture sensor and a suitable solenoid valve, which converts electrical energy into mechanical energy and, in turn, opens or closes the valve mechanically.

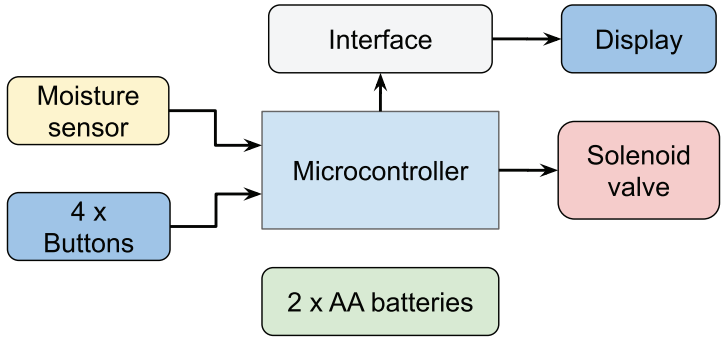

Figure 12.1 First proposal of the hardware modules of the irrigation system.

In the prototype implementation, it is reasonable to use the NUCLEO board for the microcontroller module. The reader already owns this board and knows that it is capable of implementing all the software functionality.

For the display, any of the options introduced in Chapter 6 can be used. For the sake of simplicity, and considering the hardware costs, the character-based LCD display is selected using 4-bit mode.

The buttons will be implemented using a breadboard and tactile switches, as in Chapter 1. Therefore, all that is needed is to define how to implement the moisture sensor, the solenoid valve, and the battery power supply.

In Table 12.10, three moisture sensor modules are shown. After this comparison, it seems reasonable to use the HL-69 in this first prototype due to its price. It also seems very difficult to accomplish Requirement 5.1, related to having an accuracy of better than 5%, as accuracy is not specified for any of the sensors.

Table 12.10 Comparison of moisture sensors.

Sensor name	Technology	Accuracy	Interface	Unit price [USD]
HL-69	Resistive	Not specified	VCC, GND, Digital Output, Analog Output	2
SEN-13322	Resistive	Not specified	VCC, GND, Analog Output	5
Moisture v1.2	Capacitive	Not specified	VCC, GND, Analog Output	2

In Table 12.11, three solenoid valves are shown. After this comparison, it seems reasonable to use an FPD-270A in the first prototype. Given that the solenoid valve must be powered using 12 V, a relay module will need to be used to control its activation. In order to keep this first design simple, a 12 V power supply can be included in the system. Consequently, in this prototype, the NUCLEO board can be supplied using the USB connection, as it has throughout this book.

NOTE: In a future version of the system, it might be considered to use a switching step-up power supply connected to two AA batteries to provide 12 V for the solenoid and 5 V for the NUCLEO board.

Table 12.11 Comparison of solenoid valves.

Solenoid name	Water-in connector	Activation method	Unit price [USD]
FPD-270A	½-inch	12 V / 0.25 A	9
USS-NSV00003	½-inch	12 V / 1 A	35
VA-8H	½-inch	12 V / 0.5 A	45

The resulting final version of the hardware modules of the irrigation system is shown in Figure 12.2.

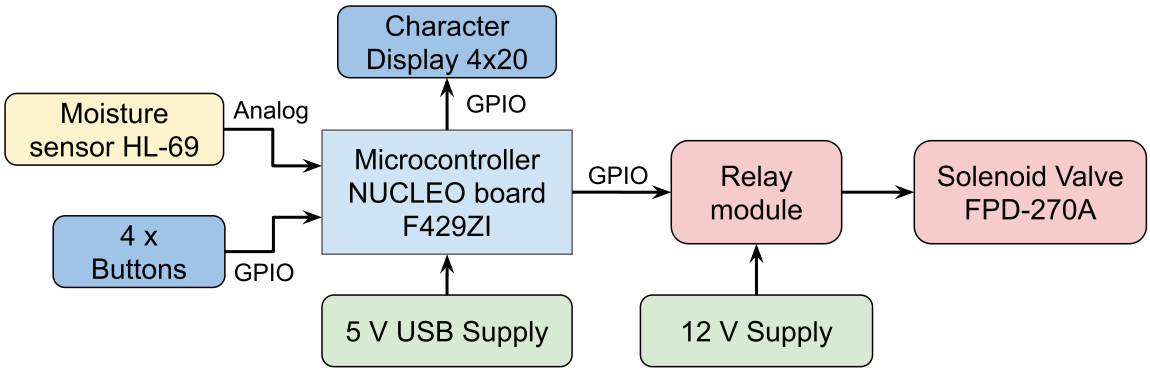

Figure 12.2 Final version of the hardware modules of the irrigation system.

NOTE: In the hardware design shown in Figure 12.2, an MB102 module is not used because the 300 mA maximum current consumption from the 5 V USB supply discussed in Chapter 4 is not reached.

The proposed connection diagram of all the hardware elements is shown in Figure 12.3. From Table 12.12 to Table 12.17, the corresponding connections are summarized. For the pin assignments, the diagram available in [1] has been used, while the connections used in previous chapters were kept whenever possible.

Table 12.12 Summary of the connections between the NUCLEO board and the character-based LCD display.

NUCLEO board	Character LCD display
D4	D4
D5	D5
D6	D6
D7	D7
D8	RS
D9	E

Table 12.13 Summary of other connections that should be made to the character-based LCD display.

Character LCD display	Voltage/Element
VSS	GND
VDD	5 V
VO	10 kΩ potentiometer
R/W	GND
A	1 kΩ resistor to 5 V
K	GND

Table 12.14 Summary of the connections between the NUCLEO board and the buttons.

NUCLEO board	Button
PG_1	"Mode"
PF_9	"How Often"
PF_7	"How Long"
PF_8	"Moisture"

Table 12.15 Summary of connections that should be made to the HL-69 moisture sensor.

HL-69 moisture sensor	Voltage/Element
GND	GND
VCC	3.3 V
DO	Unconnected
AO	A3 pin - NUCLEO board

Table 12.16 Summary of connections that should be made to the relay module.

Relay module	Voltage/Element
VCC	5 V
GND	GND
IN1	PF_2
NO1	FPD-270A (Terminal 1)
COM1	12 V
NC1	Unconnected
IN2	Unconnected
NO2	Unconnected
COM2	Unconnected
NC2	Unconnected

Table 12.17 Summary of connections that should be made to the FPD-270A.

FPD-270A	Voltage/Element
Terminal 1	Relay module (NO1)
Terminal 2	GND12

NOTE: The GND terminal of the 12 V power supply (named *GND12*) was intentionally kept isolated from the GND of the 5 V USB power supply. This is done to prevent any potential damage to the microcontroller caused by the 12 V voltage, and also to diminish the electrical noise interference over the microcontroller that could be generated when the load is activated, as was explained in Chapter 7.

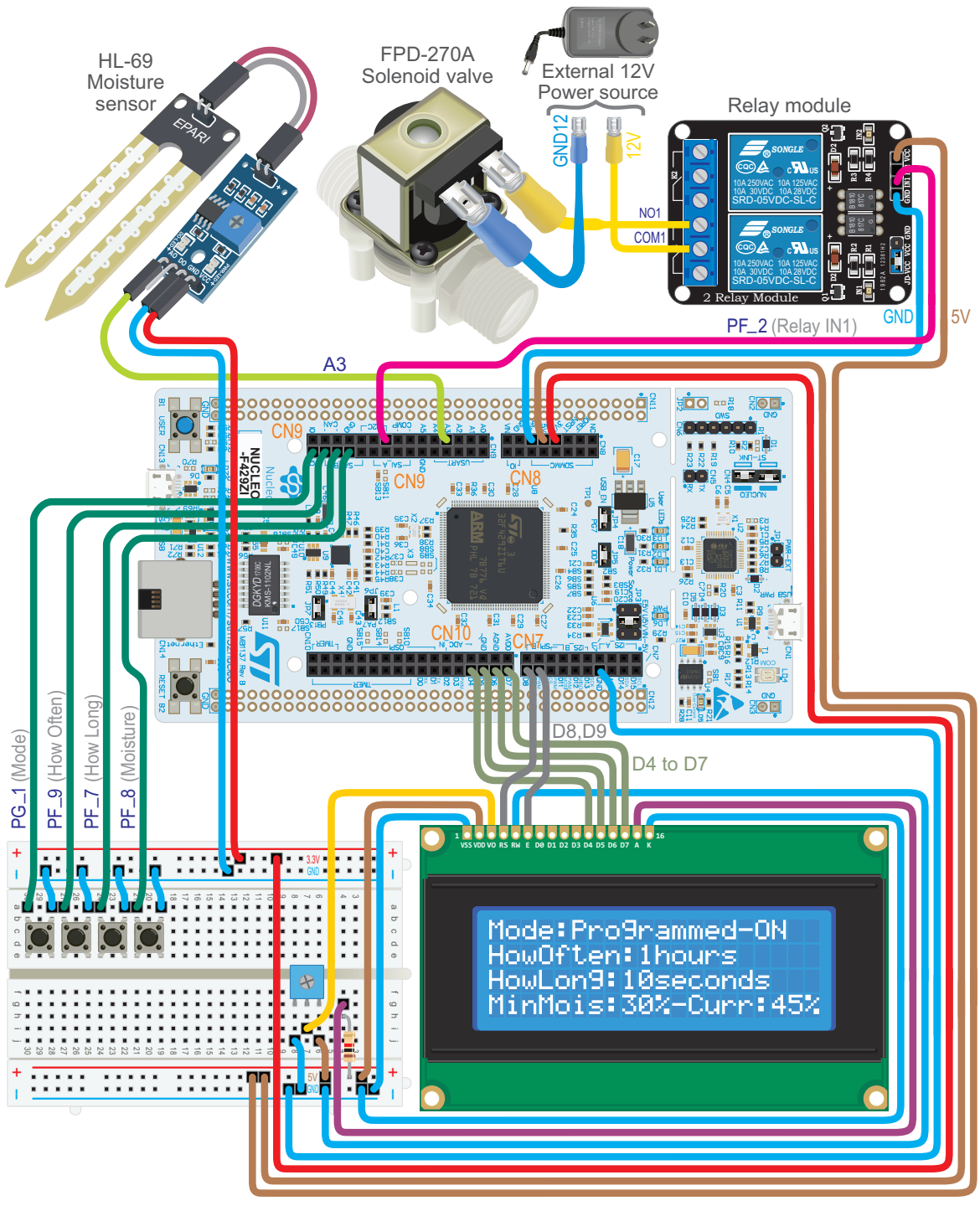

Figure 12.3 Connection diagram of all the hardware elements of the irrigation system.

NOTE: The connections used in previous chapters have been kept whenever possible. In this way, the setup shown in Figure 12.3 can be promptly implemented from the setup used in previous chapters.

TIP: In order to be able to use the program codes of Chapters 6 to 11 again without delay, the elements connected in those chapters can be left connected to the NUCLEO board and to the breadboard (they will not cause any interference in this chapter). In that case, disconnect the wire that connects the 3.3 V output of the NUCLEO board with the breadboard, and use the MB102 module to supply 3.3 V to the system.

Finally, in Table 12.18, the bill of materials is shown. It can be seen that the estimated cost is below 60 USD, and therefore requirement 8.1, which was established in Table 12.6, is fulfilled. Note that if this prototype were to be produced in quantity, a redesign would have to be done. This would lower some costs (e.g., a bespoke design of the microcontroller board will save cost), while it would increase other costs (e.g., container, printed circuit board, time required).

Table 12.18 Bill of materials.

Item	Quantity	Price [USD]
NUCLEO Board F429ZI	1	25
Moisture sensor HL-69	1	2
FPD-270A solenoid valve	1	9
Character display 4 × 20	1	15
Relay module	1	5
Tactile switches	4	0.1
	Total:	56.4

Proposed Exercise

1. Design the hardware of the project that was selected in the "Proposed Exercise" of Example 12.1. Create a diagram of the connections and tables indicating all the details. A bill of materials will also be very useful.

Answer to the Exercise

1. It is strongly recommended to start by reusing as much as possible from the previous chapters of this book. A search of the internet may help to find appropriate components and circuits for the remaining parts of the project.

Example 12.4: Design the Software

Objective

Design software that fulfills the functionality described in the requirements and the use cases.

Summary of the Expected Outcome

The results of this step are expected to include:

- a diagram of the software modules indicating all their interconnections,
- a table indicating the variables and objects of each of the software modules,
- a table indicating the functions of each of the software modules that will be used, and
- a diagram of the finite-state machine (FSM) that will be used to implement the functionality.

Discussion of How to Implement this Step

Given the set of requirements, the software design is not necessarily unique. The design will depend on the developer's experience and preferences. A reasonable approach to this step might be to start by analyzing designs made by other developers to implement similar requirements. This also includes the previous chapters of this book. A diagram using the ideas shown in Chapter 5 can be the starting point. After this, a set of tables showing the variables, objects, and functions of each of the software modules can be prepared. Finally, a diagram of the FSM that will be used to implement the functionality can be drawn, as well as a sketch of the proposed layout for the display.

Implementation of this Proposed Step

Since the functionality of the system does not require any high-speed reactions, a very simple approach such as the one shown in Figure 12.4 can be followed. It can be seen that there is an initialization of all the modules and then an update every 100 milliseconds. The reader might notice that it is very similar to the approach followed in Chapter 5 of this book. However, a non-blocking delay will be used to obtain more accurate timing behavior.

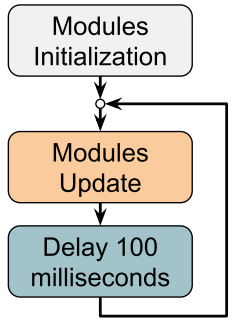

Figure 12.4 Software design of the irrigation system program.

The next step will be to determine the software modules. Figure 12.5 shows a proposal based on

software modules introduced in previous chapters (display, relay, etc.), as well as the assumption that it will be possible to implement a *Moisture sensor* module. An *Irrigation timer* module has been included to account for the *waiting time* between irrigations, and to account for the *irrigation time* during the irrigation. Finally, an *Irrigation control* module has been included to implement an FSM to control the irrigation.

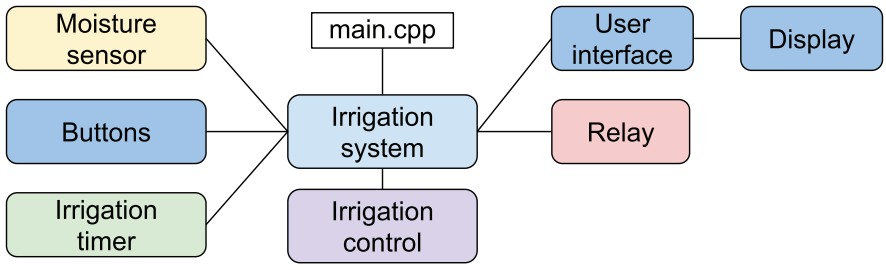

Figure 12.5 Software modules of the irrigation system program.

The proposed organization of folders and files to implement the software is shown in Figure 12.6. It can be appreciated that it is very similar to the organization introduced in Chapter 5.

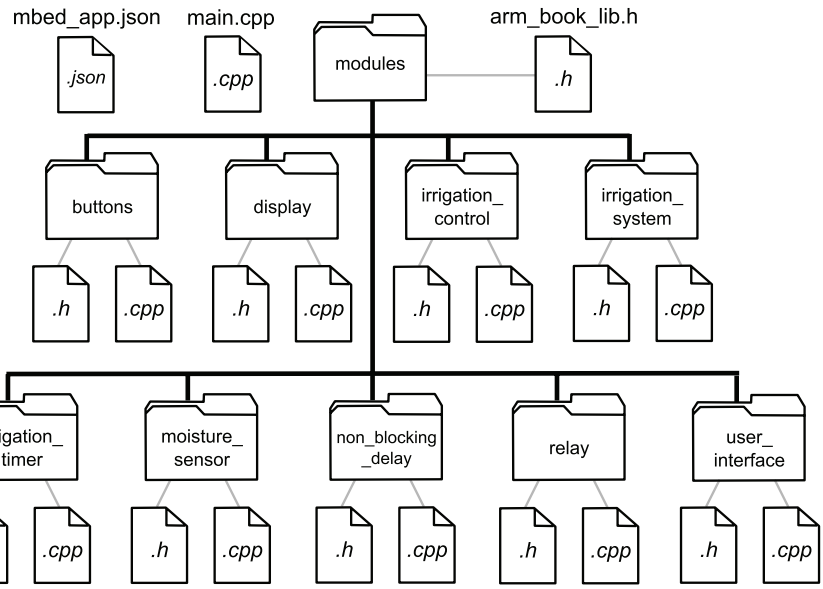

Figure 12.6 Diagram of the .cpp and .h files of the irrigation system software.

In Table 12.19, there is a brief description of each of the modules shown in Figure 12.5. Note that the proposed implementation is very simple, with the aim of keeping the attention on the proposed steps to implement a project. For this reason, there is only one driver that is used to manage the display, which uses the same files introduced in Chapter 6 and will be used without any modification.

Table 12.19 Functionalities and roles of the home irrigation system modules.

Module	Description of its functionality	Role
irrigation_system	Calls initialization and update functions of other modules	System
moisture_sensor	Reads the HL-69 moisture sensor and processes the readings	Subsystem
buttons	Detects buttons pressed and accounts for the configurations	Subsystem
irrigation_timer	Accounts for waiting and irrigation time in programmed mode	Subsystem
irrigation_control	Controls irrigation based on the mode and the timer	Subsystem
relay	Controls relay activation	Subsystem
user_interface	Sends information to be printed to the display driver	Subsystem
display	Receives commands from the user_interface module	Driver

NOTE: In the proposed implementation, the *buttons* module detects the buttons that are pressed by the user. It also processes the information to determine the current value of the configurations that were detailed in Table 12.6: "How often", "How long", and "Minimum moisture level". In addition, it detects when the Mode button is pressed and informs the *irrigation_control* module, which determines the current operation mode.

The proposed next step is to define the private variables and objects of each of the subsystem modules. These are shown in Table 12.20 to Table 12.25.

Table 12.20 Private objects and variables of the moisture_sensor module.

Name of the element	Type	Description of its functionality
hl69	AnalogIn object	Is used to read the A3 analog input of the NUCLEO board where the HL-69 is connected.
hl69AveragedValue	Float variable	Is used to process the reading to avoid noise problems. It is the average of the last ten readings.
hl69ReadingsArray	Float variable	Is used to store the last ten readings of the HL-69 moisture sensor.

Table 12.21 Private objects and variables of the buttons module.

Name of the element	Type	Description of its functionality
changeModeButton	DigitalIn object	Is used to read the PG_1 digital input of the NUCLEO board, which indicates when the current mode has to be changed.
howOftenButton	DigitalIn object	Is used to read the PF_9 digital input of the NUCLEO board, by means of which the "how often" configuration is changed.
howLongButton	DigitalIn object	Is used to read the PF_7 digital input of the NUCLEO board, by means of which the "how long" configuration is changed.
moistureButton	DigitalIn object	Is used to read the PF_8 digital input of the NUCLEO board, by means of which the "minimum moisture level" is changed.
buttonsStatus	Typedef	Is used to store the configuration status. Its members are changeMode, howOften, howLong, and moisture.

Table 12.22 Private objects and variables of the irrigation_timer module.

Name of the element	Type	Description of its functionality
irrigationTimer	Typedef	Used to track the status of the timers. Its members are *waitedTime* and *irrigatedTime*, both integer variables.

Table 12.23 Private objects and variables of the irrigation_control module.

Name of the element	Type	Description of its functionality
irrigationControlStatus	Typedef	Used to inform the state of the FSM that is implemented in this module and also used to indicate when to reset the timers of the irrigation_timer module. Its members are: • *irrigationState:* Enum type defined, with valid states: INITIAL_MODE_ASSESSMENT, CONTINUOUS_MODE_IRRIGATING, PROGRAMMED_MODE_WAITING_TO_IRRIGATE, PROGRAMMED_MODE_IRRIGATION_SKIPPED, and PROGRAMMED_MODE_IRRIGATING. • *waitedTimeMustBeReset:* a Boolean variable. • *irrigatedTimeMustBeReset:* a Boolean variable.

Table 12.24 Private objects and variables of the user_interface module.

Name of the element	Type	Description of its functionality
–	–	This module has no private objects or variables.

Table 12.25 Private objects and variables of the relay module.

Name of the element	Type	Description of its functionality
relayControlPin	DigitalInOut object	Used to write the PF_2 pin of the NUCLEO board. When it is 0, the relay is activated.

From Table 12.26 to Table 12.32, the proposed public functions for each of the modules are detailed.

Table 12.26 Public functions of the irrigation_system module.

Name of the function	Description of its functionality	File that uses it
irrigationSystemInit()	Initializes the subsystems of the irrigation system and the non-blocking delay.	main.cpp
irrigationSystemUpdate()	Calls the functions that update the modules when the corresponding time of non-blocking delay has elapsed.	main.cpp

Table 12.27 Public functions of the moisture_sensor module.

Name of the function	Description of its functionality	Modules that use it
moistureSensorInit()	Has no functionality.	–
moistureSensorUpdate()	Updates the value of *hl69ProcessedValue*.	irrigation_system
moistureSensorRead()	Returns *hl69ProcessedValue*.	irrigation_control user_interface

Table 12.28 Public functions of the buttons module.

Name of the function	Description of its functionality	Modules that use it
buttonsInit()	Configures all buttons in pull-up mode and sets initial configuration of the system after power on.	irrigation_system
buttonsUpdate()	Updates the values of *buttonsStatus*.	irrigation_system
buttonsRead()	Returns the values of *buttonsStatus*.	irrigation_control user_interface

Table 12.29 Public functions of the irrigation_timer module.

Name of the function	Description of its functionality	Modules that use it
irrigationTimerInit()	Sets initial values of *irrigationTimer*.	irrigation_system
irrigationTimerUpdate()	Updates the values of *irrigationTimer*.	irrigation_system
irrigationTimerRead()	Returns the values of *irrigationTimer*.	irrigation_control

Table 12.30 Public functions of the irrigation_control module.

Name of the function	Description of its functionality	Modules that use it
irrigationControlInit()	*IrrigationState* is set to "INITIAL_MODE_ASSESSMENT"; *waitedTimeMustBeReset* and *waitedTimeMustBeReset* are set to true.	irrigation_system
irrigationControlUpdate()	Updates the FSM of the *irrigation_control* module.	irrigation_system
irrigationControlRead()	Returns the values of *irrigationControlStatus*.	user_interface irrigation_timer relay

Table 12.31 Public functions of the display module.

Name of the function	Description of its functionality	Modules that use it
userInterfaceInit()	Calls *displayInit()* and prints the text that does not change over time on the display.	irrigation_system
userInterfaceUpdate()	Updates the current values of mode and configurations in the display by means of the display driver.	irrigation_system
userInterfaceRead()	Has no functionality.	-

Table 12.32 Public functions of the relay module.

Name of the function	Description of its functionality	Modules that use it
relayInit()	Initializes the value of *relayControlPin*.	irrigation_system
relayUpdate()	Updates the value of *relayControlPin*.	irrigation_system
relayRead()	Has no functionality.	-

In Figure 12.7, the proposed FSM is shown. After the start, the first state is INITIAL_MODE_ASSESSMENT. In this mode, the state of the Mode button is read (it is indicated as the *changeMode* variable in Figure 12.7). If it is pressed (*changeMode* is true), *irrigationControlStatus.irrigationState* (introduced in Table 12.23) is set to CONTINUOUS_MODE_IRRIGATING. If the Mode button is not being pressed (*changeMode* is false), then *irrigationControlStatus.irrigationState* is set to PROGRAMMED_MODE_WAITING_TO_IRRIGATE, and *irrigationControlStatus.waitedTimeMustBeReset* (Table 12.23) is set to true.

In the CONTINUOUS_MODE_IRRIGATING state, the Mode button is checked. If it is not pressed (*changeMode* is false), then *irrigationControlStatus.irrigationState* is not modified, and the FSM remains in the CONTINUOUS_MODE_IRRIGATING state. If the Mode button is pressed (*changeMode* is true), *irrigationControlStatus.irrigationState* is set to PROGRAMMED_MODE_WAITING_TO_IRRIGATE, and *irrigationControlStatus.waitedTimeMustBeReset* is set to true.

NOTE: The relay is controlled by the *relay* module based on the return value of *irrigationControlRead()*. If the read state is CONTINUOUS_MODE_IRRIGATING or PROGRAMMED_MODE_IRRIGATING, the relay will be activated, and it will be deactivated if the read state is INITIAL_MODE_ASSESSMENT, PROGRAMMED_MODE_WAITING_TO_IRRIGATE, or PROGRAMMED_MODE_IRRIGATION_SKIPPED.

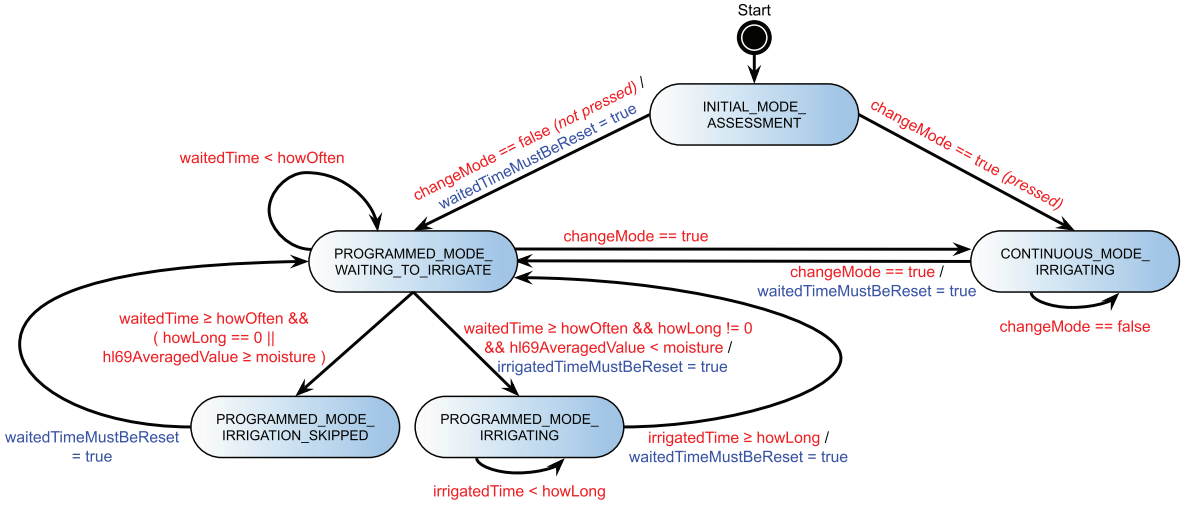

Figure 12.7 Diagram of the proposed FSM.

In the state PROGRAMMED_MODE_WAITING_TO_IRRIGATE, the first step is to set *irrigationControlStatus.waitedTimeMustBeReset* to false, because once in this state *irrigationTimer.waitedTime* (introduced in Table 12.22) has already been reset by the *irrigation_timer* module (see the Note below). Next, it is assessed whether the Mode button is pressed.

If it is (*changeMode* is true), *irrigationControlStatus.irrigationState* is set to CONTINUOUS_MODE_IRRIGATING. If the Mode button is not pressed, it is checked if *irrigationTimer.waitedTime* is smaller than *howOften*. If so, the FSM remains in PROGRAMMED_MODE_WAITING_TO_IRRIGATE. If *irrigationTimer.waitedTime* is equal to or greater than *howOften*, then the other conditions are checked.

NOTE: In the code to be implemented, if the FSM is in PROGRAMMED_MODE_WAITING _TO_IRRIGATE, the value of *irrigationTimer.waitedTime* will be increased by the *irrigation_timer* module each time *irrigationTimerUpdate()* is executed, and will be reset by the *irrigation_timer* module when it detects, by means of the return value of *irrigationControlRead()*, that *irrigationControlStatus.waitedTimeMustBeReset* is true. If the current state is PROGRAMMED_MODE_IRRIGATING, the *irrigation_timer* module will increase *irrigationTimer.irrigatedTime* (introduced in Table 12.22) each time *irrigationTimerUpdate()* is executed and will reset *irrigationTimer.irrigatedTime* if it detects that *irrigationControlStatus.irrigatedTimeMustBeReset* is true, by means of the return value of *irrigationControlRead()*. In this way, the modularization principle will not be violated, because only the *irrigation_timer* module will modify the irrigation and waiting timers.

If *hl69AveragedValue* is greater than or equal to the minimum moisture level configuration (*moisture*) or *irrigationtimer.howLong* is zero, then *irrigationControlStatus.irrigationState* is set to PROGRAMMED_MODE_IRRIGATION_SKIPPED. If *howLong* is not zero and *hl69AveragedValue* is smaller than the minimum moisture level configuration (*moisture*), then *irrigationControlStatus.irrigationState* is set to PROGRAMMED_MODE_IRRIGATING, and *irrigationControlStatus.irrigatedTimeMustBeReset* is set to true.

The state PROGRAMMED_MODE_IRRIGATION_SKIPPED is only used to show on the display that the irrigation was skipped. This is done by the *user_interface* module, which reads the value of *irrigationControlStatus.irrigationState* using *irrigationControlRead()*. In this state, *irrigationControlStatus.waitedTimeMustBeReset* is set to true, and *irrigationControlStatus.irrigationState* is set to PROGRAMMED_MODE_WAITING_TO_IRRIGATE.

In the state PROGRAMMED_MODE_IRRIGATING, *irrigationControlStatus.irrigatedTimeMustBeReset* is first set to false. Then, it assesses if *irrigationTimer.irrigatedTime* is smaller than *howLong*. After the irrigation is completed, it sets *irrigationControlStatus.waitedTimeMustBeReset* to true and *irrigationControlStatus.IrrigationState to* PROGRAMMED_MODE_WAITING_TO_IRRIGATE.

To conclude this step, in Figures 12.8 to 12.11 some sketches of the proposed layout of the LCD display are shown. On the first line of the display, the current irrigation mode is shown. It is also indicated whether the system is irrigating or not in the case of the Programmed irrigation mode. On the second and third lines of the display, the values of the “How Often” and “How Long” configurations are shown. On the fourth line, the minimum moisture level below which irrigation will be activated in the Programmed irrigation mode is shown on the left, and the current moisture measurement is shown on the right.

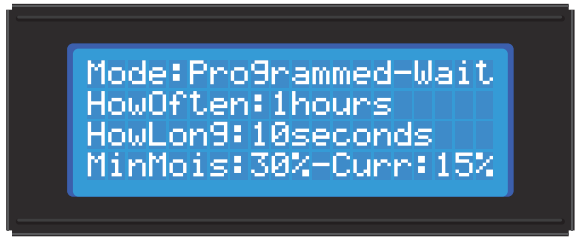

Figure 12.8 Layout of the LCD for the Programmed irrigation mode when the system is waiting to irrigate.

Figure 12.9 Layout of the LCD for the Programmed irrigation mode when the system is irrigating.

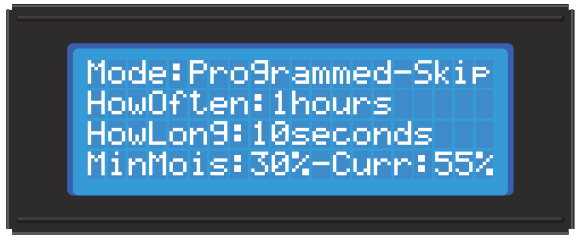

Figure 12.10 Layout of the LCD for the Programmed irrigation mode when irrigation is skipped.

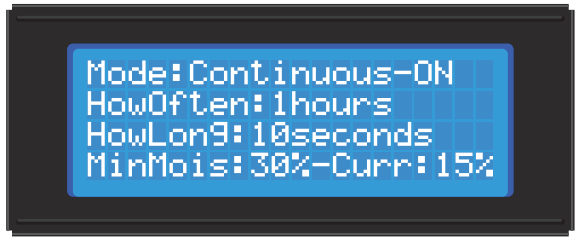

Figure 12.11 Layout of the LCD for the Continuous irrigation mode.

Proposed Exercise

1. Design the software for the project that was selected in the "Proposed Exercise" of Example 12.1. Produce a diagram of the module interconnections and tables indicating all of the details. Include a diagram of the FSM designed to implement the functionality.

Answer to the Exercise

1. It is strongly recommended to start by reusing as much as possible from the previous chapters of this book. Using the internet may help to find more ideas for the remaining parts of the software.

Example 12.5: Implement the User Interface

Objective

Implement the system's user interfaces, as designed in step 4.

Summary of the Expected Outcome

The result of this step is expected to be a set of folders and *.h* and *.cpp* files that implement the user interface.

Discussion on how to Implement this Step

In this step, the software should be implemented as a set of *.h* and *.cpp* files. The layout of the software should follow the design that was created previously unless there are strong rationales to introduce changes. In the irrigation system, the user interface consists of four buttons and a character-based LCD display. Therefore, only the corresponding functionality is implemented in this step.

Implementation of this Proposed Step

In Code 12.1, *main.cpp* is shown. On line 3, the library *irrigation_system.h* is included. The *main()* function is presented on lines 7 to 14. On line 9, the function *irrigationSystemInit()* is called, and on lines 10 to 13 the *superloop* is implemented. The function *irrigationSystemUpdate()* is called inside the superloop.

```
1  //=====[Libraries]=============================================================
2
3  #include "irrigation_system.h"
4
5  //=====[Main function, the program entry point after power on or reset]========
6
7  int main()
8  {
9      irrigationSystemInit();
10     while (true) {
11         irrigationSystemUpdate();
12     }
13 }
14
```

Code 12.1 Implementation of main.cpp.

In Code 12.2, the implementation of *irrigation_system.h* is shown. On line 8, SYSTEM_TIME_INCREMENT_MS is defined, and the public functions *irrigationSystemInit()* and *irrigationSystemUpdate()* are declared.

```
1  //=====[#include guards - begin]===============================================
2
3  #ifndef _IRRIGATION_SYSTEM_H_
4  #define _IRRIGATION_SYSTEM_H_
5
6  //=====[Declaration of public defines]==========================================
7
8  #define SYSTEM_TIME_INCREMENT_MS   100
9
10 //=====[Declarations (prototypes) of public functions]=========================
11
12 void irrigationSystemInit();
13 void irrigationSystemUpdate();
14
15 //=====[#include guards - end]=================================================
16
17 #endif // _IRRIGATION_SYSTEM_H_
```

Code 12.2 Implementation of irrigation_system.h.

In Code 12.3, the implementation of *irrigation_system.cpp* is shown. From lines 3 to 7, libraries are included. On line 11, the private variable *irrigationSystemDelay* of type *nonBlockingDelay_t* is declared. This variable will be used to implement the non-blocking delay. On line 15, the function *irrigationSystemInit()* is implemented. First, *tickInit()* is called. Then, *buttonsInit()* and *userInterfaceInit()* are called. Finally, on line 20, the non-blocking delay is initialized to SYSTEM_TIME_INCREMENT_MS.

On line 23, *irrigationSystemUpdate()* is implemented. Line 25 assesses whether *irrigationSystemDelay* has reached the value set by *nonBlockingDelayInit()*. In that case, *buttonsUpdate()* and *userInterfaceUpdate()* are called.

NOTE: For the sake of brevity, only the file sections that have some content are shown in the Codes. The full versions of the files are available in [2].

```
1  //=====[Libraries]=============================================================
2
3  #include "irrigation_system.h"
4
5  #include "buttons.h"
6  #include "user_interface.h"
7  #include "non_blocking_delay.h"
8
9  //=====[Declaration and initialization of public global variables]=============
10
11 static nonBlockingDelay_t irrigationSystemDelay;
12
13 //=====[Implementations of public functions]===================================
```

```
14
15  void irrigationSystemInit()
16  {
17      tickInit();
18      buttonsInit();
19      userInterfaceInit();
20      nonBlockingDelayInit( &irrigationSystemDelay, SYSTEM_TIME_INCREMENT_MS );
21  }
22
23  void irrigationSystemUpdate()
24  {
25      if( nonBlockingDelayRead(&irrigationSystemDelay) ) {
26          buttonsUpdate();
27          userInterfaceUpdate();
28      }
29  }
```

Code 12.3 Implementation of irrigation_system.cpp.

In Code 12.4, the implementation of *buttons.h* is shown. From line 8 to line 16, nine definitions are introduced. They are used to implement requirements 3.1.2 to 3.1.7, as shown in Table 12.33. On line 20, the type definition of the struct named *buttonsStatus_t* is shown. This struct has four members: *changeMode*, which is used to keep track of the mode; *howOften*, which is used to keep track of the value of the "How often" configuration; *howLong*, which is used to keep track of the value of the "How long" configuration; and *moisture*, which used to keep track of the value of the "Minimum moisture level" configuration. On lines 29 to 31, the three public functions of this module are declared: *buttonsInit()*, *buttonsUpdate()*, and *buttonsRead()*. The latter is the only one that has a return value, which is of type *buttonsStatus_t*.

In Code 12.5, the implementation of *buttons.cpp* is shown. On lines 10 to 13, the private DigitalIn objects *changeModeButton*, *howOftenButton*, *howLongButton*, and *moistureButton* are declared and assigned to PG_1, PF_9, PF_7, and PF_8, respectively. On line 17, the private variable *buttonsStatus* of the type *buttonsStatus_t* is declared. On line 21, *buttonsInit()* is implemented. First, all of the buttons are configured with pull-up resistors (lines 23 to 26) and then the members of *buttonsStatus* are set to an initial value (lines 28 to 31).

NOTE: In previous chapters, objects were declared public in order to simplify the software implementation. In this chapter, objects are declared private to improve the software modularity.

```
1   //=====[#include guards - begin]===============================================
2
3   #ifndef _BUTTONS_H_
4   #define _BUTTONS_H_
5
6   //=====[Declaration of public defines]=========================================
7
8   #define HOW_OFTEN_INCREMENT   1
9   #define HOW_OFTEN_MIN         1
10  #define HOW_OFTEN_MAX         24
11  #define HOW_LONG_INCREMENT    10
```

```
12  #define HOW_LONG_MIN          0
13  #define HOW_LONG_MAX          90
14  #define MOISTURE_INCREMENT    5
15  #define MOISTURE_MIN          0
16  #define MOISTURE_MAX          95
17
18  //=====[Declaration of public data types]======================================
19
20  typedef struct buttonsStatus {
21      bool changeMode;
22      int howOften;
23      int howLong;
24      int moisture;
25  } buttonsStatus_t;
26
27  //=====[Declarations (prototypes) of public functions]=========================
28
29  void buttonsInit();
30  void buttonsUpdate();
31  buttonsStatus_t buttonsRead();
32
33  //=====[#include guards - end]=================================================
34
35  #endif // _BUTTONS_H_
```

Code 12.4 Implementation of buttons.h.

Table 12.33 Some of the initial requirements defined for the home irrigation system.

Req. Group	Req. ID	Description
3. Configuration	3.1	The system configuration will be done by means of a set of buttons:
	3.1.1	The "Mode" button will change between "Programmed irrigation" and "Continuous irrigation".
	3.1.2	The "How often" button will increase the time between irrigations in programmed mode by one hour.
	3.1.3	The "How long" button will increase the irrigation time in programmed mode by ten seconds.
	3.1.4	The "Moisture" button will increase the "Minimum moisture level" configuration by 5%.
	3.1.5	The maximum values are: "How often": 24 h; "How long": 90 s; "Moisture": 95%.
	3.1.6	"How long" and "Moisture" will be set to 0 (zero) if they reach the maximum and the button is pressed.
	3.1.7	"How often" will be set to 1 if it reaches its maximum value and the button is pressed.

On line 34, the function *buttonsUpdate()* is implemented. First, *buttonsStatus.changeMode* is assigned the value of *!changeModeButton* because *changeModeButton* is configured with a pull-up resistor. On line 38, *howOftenButton* is assessed. If it is pressed (*howOftenButton* is false), then *buttonsStatus.howOften* is incremented by HOW_OFTEN_INCREMENT in line 39. On line 40, it is assessed if *buttonsStatus.howOften* is greater than or equal to HOW_OFTEN_MAX + HOW_OFTEN_INCREMENT. If so, it is assigned HOW_OFTEN_MIN on line 41.

The code on lines 45 to 50, and the code on lines 52 to 57, are very similar to the code from lines 38 to 43 and, therefore, are not discussed line by line.

Finally, on lines 60 to 63, the function *buttonsRead()* is implemented. This function returns the value of the private variable *buttonsStatus*. In this way, other modules can get the values of the

buttonsStatus members *changeMode*, *howOften*, *howLong*, and *moisture* without violating the principle of modularization.

```
//=====[Libraries]=============================================================

#include "mbed.h"
#include "arm_book_lib.h"

#include "buttons.h"

//=====[Declaration and initialization of private global objects]===============

static DigitalIn changeModeButton(PG_1);
static DigitalIn howOftenButton(PF_9);
static DigitalIn howLongButton(PF_7);
static DigitalIn moistureButton(PF_8);

//=====[Declaration and initialization of private global variables]============

static buttonsStatus_t buttonsStatus;

//=====[Implementations of public functions]===================================

void buttonsInit()
{
    changeModeButton.mode(PullUp);
    howOftenButton.mode(PullUp);
    howLongButton.mode(PullUp);
    moistureButton.mode(PullUp);

    buttonsStatus.changeMode = OFF;
    buttonsStatus.howOften = HOW_OFTEN_MIN;
    buttonsStatus.howLong = HOW_LONG_MIN;
    buttonsStatus.moisture = MOISTURE_MIN;
}

void buttonsUpdate()
{
    buttonsStatus.changeMode = !changeModeButton;

   if ( !howOftenButton ) {
        buttonsStatus.howOften = buttonsStatus.howOften + HOW_OFTEN_INCREMENT;
        if ( buttonsStatus.howOften >= HOW_OFTEN_MAX + HOW_OFTEN_INCREMENT) {
            buttonsStatus.howOften = HOW_OFTEN_MIN;
        }
   }

   if ( !howLongButton ) {
        buttonsStatus.howLong = buttonsStatus.howLong + HOW_LONG_INCREMENT;
        if ( buttonsStatus.howLong >= HOW_LONG_MAX + HOW_LONG_INCREMENT) {
            buttonsStatus.howLong = HOW_LONG_MIN;
        }
   }

   if ( !moistureButton ) {
        buttonsStatus.moisture = buttonsStatus.moisture + MOISTURE_INCREMENT;
        if ( buttonsStatus.moisture >= MOISTURE_MAX + MOISTURE_INCREMENT) {
           buttonsStatus.moisture = MOISTURE_MIN;
        }
   }
}

buttonsStatus_t buttonsRead()
{
    return buttonsStatus;
}
```

Code 12.5 Implementation of buttons.cpp.

In Code 12.6, the implementation of *user_interface.h* is shown. The private functions *userInterfaceInit()*, *userInterfaceUpdate()*, and *userInterfaceRead()* are declared on lines 8 to 10.

```
1  //=====[#include guards - begin]===============================================
2
3  #ifndef _USER_INTERFACE_H_
4  #define _USER_INTERFACE_H_
5
6  //=====[Declarations (prototypes) of public functions]=========================
7
8  void userInterfaceInit();
9  void userInterfaceUpdate();
10 void userInterfaceRead();
11
12 //=====[#include guards - end]=================================================
13
14 #endif // _USER_INTERFACE_H_
```

Code 12.6 Implementation of user_interface.h.

In Code 12.7, the first part of the implementation of *user_interface.cpp* is shown. On lines 3 to 9, the libraries are included. From lines 13 to 27, the function *userInterfaceInit()* is implemented. On line 15, *displayInit()* is used to establish that a character-based display in 4-bit mode is used. On line 17, *displayClear()* is used to clear the display. Lines 19 to 25 are used to write some text on the display following the design introduced in Figure 12.8. This text does not change in this example, even if the buttons are pressed.

```
1  //=====[Libraries]=============================================================
2
3  #include "mbed.h"
4  #include "arm_book_lib.h"
5
6  #include "user_interface.h"
7
8  #include "display.h"
9  #include "buttons.h"
10
11 //=====[Implementations of public functions]===================================
12
13 void userInterfaceInit()
14 {
15     displayInit( DISPLAY_TYPE_LCD_HD44780,DISPLAY_CONNECTION_GPIO_4BITS );
16
17     displayClear();
18     displayCharPositionWrite( 0, 0 );
19     displayStringWrite( "Mode:Programmed-Wait" );
20     displayCharPositionWrite( 0, 1 );
21     displayStringWrite( "HowOften:  hours" );
22     displayCharPositionWrite( 0, 2 );
23     displayStringWrite( "HowLong:  seconds" );
24     displayCharPositionWrite( 0, 3 );
25     displayStringWrite( "MinMois:  %-Curr:15%" );
26 }
```

Code 12.7 Implementation of user_interface.cpp (Part 1/2).

From lines 1 to 20 of Code 12.8, the function *userInterfaceUpdate()* is implemented. On line 3, an array of char named *number* is declared. On line 5, a variable named *buttonsStatusLocalCopy* of type *buttonsStatus_t* is declared. On line 7, *buttonsStatusLocalCopy* is assigned the return value of *buttonsRead()*. Lines 9 to 11 are used to write the value of *buttonsStatusLocalCopy.howOften* to location (9,1) of the display. The format tag prototype "%02d" is used to indicate that two characters should be used for the value. If the value to be written has less than two characters, the result is padded with leading zeros. A very similar approach is used to write *buttonsStatusLocalCopy.howLong* (lines 13 to 15) and *buttonsStatusLocalCopy.moisture* (lines 17 to 19). Lastly, on line 22, it can be seen that *userInterfaceRead()* has no functionality.

```
 1  void userInterfaceUpdate()
 2  {
 3      char number[3];
 4
 5      buttonsStatus_t buttonsStatusLocalCopy;
 6
 7      buttonsStatusLocalCopy = buttonsRead();
 8
 9      displayCharPositionWrite( 9, 1 );
10      sprintf( number, "%02d", buttonsStatusLocalCopy.howOften );
11      displayStringWrite( number );
12
13      displayCharPositionWrite( 8, 2 );
14      sprintf( number, "%02d", buttonsStatusLocalCopy.howLong );
15      displayStringWrite( number );
16
17      displayCharPositionWrite( 8, 3 );
18      sprintf( number, "%02d", buttonsStatusLocalCopy.moisture );
19      displayStringWrite( number );
20  }
21
22  void userInterfaceRead()
23  {
24  }
```

Code 12.8 Implementation of user_interface.cpp (Part 2/2).

Finally, given that the pins used to connect the display (Table 12.12) are the same pins used in Chapter 6, it is not necessary to modify the pins assignment used in *display.cpp*. If the display were to be connected to other pins, then *displayEN*, *displayRS*, *displayD4*, *displayD5*, *displayD6*, and *displayD7* would have to be assigned with the corresponding pins.

Proposed Exercise

1. Implement the system's user interface for the project that was selected in the "Proposed Exercise" of Example 12.1. Write all the corresponding *.h* and *.cpp* files.

Answer to the Exercise

1. It is strongly recommended to start by reusing as much as possible from the previous chapters of this book, or even from this example.

Example 12.6: Implement the Reading of the Sensors

Objective

Implement the reading of the sensors, as designed in step 4.

Summary of the Expected Outcome

As a result of this step, it is expected to have a set of *.h* and *.cpp* files that implement the user interface and the reading of the sensors.

Discussion on How to Implement this Step

As was previously mentioned, the layout of the software should follow the design that was done in step 4 unless there are strong rationales to introduce changes. The irrigation system has only one sensor, the moisture sensor. Therefore, only the corresponding functionality is implemented in this step.

Implementation of this Proposed Step

For the sake of brevity, in this example only new files or files that have changes are shown. The full set of files for this example are available in [2]. In particular, new lines are introduced in *irrigation_system.cpp*, as can be seen in Code 12.9. The library *moisture_sensor.h* is included on line 8, and the functions *moistureSensorInit()* and *moistureSensorUpdate()* are called on lines 21 and 30, respectively.

There are also a few new lines in *user_interface.cpp*, as can be seen in Code 12.10. On line 10, the library *moisture_sensor.h* is included. On line 26, "MinMois: %-Curr: %" is written. On line 34, the float variable *hl69AveragedValueLocalCopy* is declared, and on line 37 it is assigned the return value of *moistureSensorRead()*. Lines 51 to 53 are included in order to write the reading of the moisture sensor.

```
1  //=====[Libraries]=============================================================
2
3  #include "irrigation_system.h"
4
5  #include "buttons.h"
6  #include "user_interface.h"
7  #include "non_blocking_delay.h"
8  #include "moisture_sensor.h"
9
10 //=====[Declaration and initialization of public global variables]=============
11
12 static nonBlockingDelay_t irrigationSystemDelay;
13
14 //=====[Implementations of public functions]===================================
15
16 void irrigationSystemInit()
17 {
18     tickInit();
19     buttonsInit();
20     userInterfaceInit();
```

```
21      moistureSensorInit();
22      nonBlockingDelayInit( &irrigationSystemDelay, SYSTEM_TIME_INCREMENT_MS );
23  }
24
25  void irrigationSystemUpdate()
26  {
27      if( nonBlockingDelayRead(&irrigationSystemDelay) ) {
28          buttonsUpdate();
29          userInterfaceUpdate();
30          moistureSensorUpdate();
31      }
32  }
```

Code 12.9 New implementation of irrigation_system.cpp.

```
1   //=====[Libraries]=============================================================
2
3   #include "mbed.h"
4   #include "arm_book_lib.h"
5
6   #include "user_interface.h"
7
8   #include "display.h"
9   #include "buttons.h"
10  #include "moisture_sensor.h"
11
12  //=====[Implementations of public functions]===================================
13
14  void userInterfaceInit()
15  {
16      displayInit( DISPLAY_TYPE_LCD_HD44780,DISPLAY_CONNECTION_GPIO_4BITS );
17
18      displayClear();
19      displayCharPositionWrite( 0, 0 );
20      displayStringWrite( "Mode:Programmed-Wait" );
21      displayCharPositionWrite( 0, 1 );
22      displayStringWrite( "HowOften:  hours" );
23      displayCharPositionWrite( 0, 2 );
24      displayStringWrite( "HowLong:  seconds" );
25      displayCharPositionWrite( 0, 3 );
26      displayStringWrite( "MinMois:  %-Curr:  %" );
27  }
28
29  void userInterfaceUpdate()
30  {
31      char number[3];
32
33      buttonsStatus_t buttonsStatusLocalCopy;
34      float hl69AveragedValueLocalCopy;
35
36      buttonsStatusLocalCopy = buttonsRead();
37      hl69AveragedValueLocalCopy = moistureSensorRead();
38
39      displayCharPositionWrite( 9, 1 );
40      sprintf( number, "%02d", buttonsStatusLocalCopy.howOften );
41      displayStringWrite( number );
42
43      displayCharPositionWrite( 8, 2 );
44      sprintf( number, "%02d", buttonsStatusLocalCopy.howLong );
45      displayStringWrite( number );
46
```

```
47      displayCharPositionWrite( 8, 3 );
48      sprintf( number, "%02d", buttonsStatusLocalCopy.moisture );
49      displayStringWrite( number );
50
51      displayCharPositionWrite( 17, 3 );
52      sprintf( number, "%2.0f", 100*hl69AveragedValueLocalCopy );
53      displayStringWrite( number );
54  }
55
56  void userInterfaceRead()
57  {
58  }
```

Code 12.10 New implementation of user_interface.cpp.

In Code 12.11, the implementation of *moisture_sensor.h* is shown. It can be seen that the public functions *moistureSensorInit()*, *moistureSensorUpdate()*, and *moistureSensorRead()* are declared.

```
1   //=====[#include guards - begin]===============================================
2
3   #ifndef _MOISTURE_SENSOR_H_
4   #define _MOISTURE_SENSOR_H_
5
6   //=====[Declarations (prototypes) of public functions]=========================
7
8   void moistureSensorInit();
9   void moistureSensorUpdate();
10  float moistureSensorRead();
11
12  //=====[#include guards - end]=================================================
13
14  #endif // _MOISTURE_SENSOR_H_
```

Code 12.11 Implementation of moisture_sensor.h.

In Code 12.12, the first part of the implementation of *moisture_sensor.cpp* is shown. From lines 3 to 6, the libraries are included. On line 10, NUMBER_OF_AVERAGED_SAMPLES is defined as 10. On line 14, the AnalogIn object *hl69* is declared and assigned to the A3 pin. On line 18, the private float variable *hl69AveragedValue*, which will be used to store the average of the last NUMBER_OF_AVERAGED_ SAMPLES, is declared and initialized. The private array of float variable *hl69ReadingsArray* is declared on line 19. On line 23, the implementation of the function *moistureSensorInit()* is shown. As can be seen, it has no functionality.

In Code 12.13, the implementation of the function *moistureSensorUpdate()* is shown on lines 1 to 18. Note that it is very similar to the way in which the LM35 sensor was read earlier in this book. On line 3, a static integer variable named *hl69SampleIndex* is declared. On line 4, an integer variable *i* is declared. On line 6, the result of 1 minus the reading of the HL-69 sensor is assigned to the corresponding position of *hl69ReadingsArray*. The assignment is done this way because the sensor retrieves 1 when the moisture is 0%. On lines 8 to 12, *hl69AveragedValue* is computed. On lines 14 to 17, the value of *hl69SampleIndex* is updated. Finally, the function *moistureSensorRead()* is implemented on lines 20 to 23.

```
1  //=====[Libraries]=============================================================
2
3  #include "mbed.h"
4  #include "arm_book_lib.h"
5
6  #include "moisture_sensor.h"
7
8  //=====[Declaration of private defines]========================================
9
10 #define NUMBER_OF_AVERAGED_SAMPLES    10
11
12 //=====[Declaration and initialization of private global objects]===============
13
14 static AnalogIn hl69(A3);
15
16 //=====[Declaration and initialization of private global variables]============
17
18 static float hl69AveragedValue = 0.0;
19 static float hl69ReadingsArray[NUMBER_OF_AVERAGED_SAMPLES];
20
21 //=====[Implementations of public functions]===================================
22
23 void moistureSensorInit()
24 {
25 }
```

Code 12.12 Implementation of moisture_sensor.cpp (Part 1/2).

```
1  void moistureSensorUpdate()
2  {
3      static int hl69SampleIndex = 0;
4      int i;
5
6      hl69ReadingsArray[hl69SampleIndex] = 1 - hl69.read();
7
8      hl69AveragedValue = 0.0;
9      for (i = 0; i < NUMBER_OF_AVERAGED_SAMPLES; i++) {
10         hl69AveragedValue = hl69AveragedValue + hl69ReadingsArray[i];
11     }
12     hl69AveragedValue = hl69AveragedValue / NUMBER_OF_AVERAGED_SAMPLES;
13
14     hl69SampleIndex++;
15     if ( hl69SampleIndex >= NUMBER_OF_AVERAGED_SAMPLES) {
16         hl69SampleIndex = 0;
17     }
18 }
19
20 float moistureSensorRead()
21 {
22     return hl69AveragedValue;
23 }
```

Code 12.13 Implementation of moisture_sensor.cpp (Part 2/2).

NOTE: The fact that the positions of the *hl69ReadingsArray* are not initialized and *hl69AveragedValue* is calculated using those values does not lead to incorrect behavior, as this situation lasts for only the first second after power on, during which the Programmed mode is still waiting to irrigate.

Proposed Exercise

1. Implement the reading of the sensors for the project that was selected in the "Proposed Exercise" of Example 12.1. Write all the corresponding *.h* and *.cpp* files.

Answer to the Exercise

1. It is strongly recommended to start by reusing as much as possible from the previous chapters of this book, or even from this example.

Example 12.7: Implement the Driving of the Actuators

Objective

Implement the drivers for the actuators, as designed in step 4.

Summary of the Expected Outcome

As a result of this step, it is expected to have a set of *.h* and *.cpp* files that implement the user interface, the reading of the sensors, and the drivers for the actuators.

Discussion of How to Implement this Step

The irrigation system has only one actuator, the solenoid valve, which is activated by means of a relay module. In this example, the activation of this relay module is implemented.

Implementation of this Proposed Step

New lines are introduced in *irrigation_system.cpp*, as can be seen on lines 9, 23, and 33 of Code 12.14. The other files that were introduced in previous examples are not changed.

In Code 12.15, the implementation of *relay.h* is shown. It can be seen that the public functions *relayInit()*, *relayUpdate()*, and *relayRead()* are declared.

```
1  //=====[Libraries]=============================================================
2
3  #include "irrigation_system.h"
4
5  #include "buttons.h"
6  #include "user_interface.h"
7  #include "non_blocking_delay.h"
8  #include "moisture_sensor.h"
9  #include "relay.h"
10
11 //=====[Declaration and initialization of public global variables]=============
12
13 static nonBlockingDelay_t irrigationSystemDelay;
14
```

```
15  //=====[Implementations of public functions]=================================
16
17  void irrigationSystemInit()
18  {
19      tickInit();
20      buttonsInit();
21      userInterfaceInit();
22      moistureSensorInit();
23      relayInit();
24      nonBlockingDelayInit( &irrigationSystemDelay, SYSTEM_TIME_INCREMENT_MS );
25  }
26
27  void irrigationSystemUpdate()
28  {
29      if( nonBlockingDelayRead(&irrigationSystemDelay) ) {
30          buttonsUpdate();
31          userInterfaceUpdate();
32          moistureSensorUpdate();
33          relayUpdate();
34      }
35  }
```

Code 12.14 New implementation of irrigation_system.cpp.

```
1   //=====[#include guards - begin]===============================================
2
3   #ifndef _RELAY_H_
4   #define _RELAY_H_
5
6   //=====[Declarations (prototypes) of public functions]=========================
7
8   void relayInit();
9   void relayUpdate();
10  float relayRead();
11
12  //=====[#include guards - end]=================================================
13
14  #endif // _RELAY_H_
```

Code 12.15 Implementation of relay.h.

In Code 12.16, the Implementation of *relay.cpp* is shown. On lines 3 to 8, the libraries are included. On line 12, a private DigitalInOut object named *relayControlPin* is declared and assigned PF_2. The function r*elayInit()* initializes *relayControlPin* as an input, which turns off the relay.

NOTE: This same type of initialization and use was implemented in previous chapters to control the buzzer using the relay, given that buzzers are 5 V devices, and it is not advised to turn them on directly using a digitalOut object.

```
1  //=====[Libraries]=============================================================
2
3  #include "mbed.h"
4  #include "arm_book_lib.h"
5
6  #include "relay.h"
7
8  #include "buttons.h"
9
10 //=====[Declaration and initialization of private global objects]===============
11
12 static DigitalInOut relayControlPin(PF_2);
13
14 //=====[Implementations of public functions]===================================
15
16 void relayInit()
17 {
18     relayControlPin.mode(OpenDrain);
19     relayControlPin.input();
20 }
21
22 void relayUpdate()
23 {
24     buttonsStatus_t buttonsStatusLocalCopy;
25
26     buttonsStatusLocalCopy = buttonsRead();
27
28    if( buttonsStatusLocalCopy.changeMode ) {
29         relayControlPin.output();
30         relayControlPin = LOW;
31     } else {
32             relayControlPin.input();
33     }
34 }
35
36 void relayRead()
37 {
38 }
```

Code 12.16 Implementation of relay.cpp.

The implementation of the function *relayUpdate()* is shown from lines 22 to 34. On line 24, *buttonsStatusLocalCopy*, a variable of type *buttonsStatus_t*, is declared. On line 26, *buttonsRead()* is used to load the status of the buttons into this variable. On line 28, *buttonsStatusLocalCopy.changeMode* is assessed. If it is true, then the relay is turned on by means of the statements on lines 29 and 30. Otherwise, the relay is turned off on line 32. In this way, the relay should become active each time the Mode button is pressed.

The function *relayRead()*, which has no functionality, is shown on lines 36 to 38.

NOTE: The behavior implemented in this example only has the purpose of testing the activation of the relay. The behavior will be changed in the next example as the remaining modules of the software are incorporated in the system implementation.

Proposed Exercise

1. Implement the drivers of the actuators for the project that was selected in the "Proposed Exercise" of Example 12.1. Write all the corresponding *.h* and *.cpp* files.

Answer to the Exercise

1. It is strongly recommended to start by reusing as much as possible from the previous chapters of this book, or even from this example. Remember that a quick look on the internet may help to find more ideas for the remaining parts of the software.

Example 12.8: Implement the Behavior of the System

Objective

Implement the behavior of the system as established in previous steps (Table 12.6 and Figure 12.7).

Summary of the Expected Outcome

As a result of this step, it is expected to have a set of *.h* and *.cpp* files that implement the complete behavior of the system.

Discussion on How to Implement this Step

In this step, the last two remaining modules, *irrigation_timer* and *irrigation_control*, are included in the system, and all the functionality described in the requirements is implemented.

Implementation of this Proposed Step

New lines are introduced in *irrigation_system.cpp*, as can be seen on lines 10, 11, 25, 26, 37, and 38 of Code 12.17. In this new implementation, *userInterfaceInit()* and *userInterfaceUpdate()* are called after all the other initialization and update function calls to properly implement the logic introduced in Example 12.4.

```
1  //=====[Libraries]=============================================================
2
3  #include "irrigation_system.h"
4
5  #include "buttons.h"
6  #include "user_interface.h"
7  #include "non_blocking_delay.h"
8  #include "moisture_sensor.h"
9  #include "relay.h"
10 #include "irrigation_control.h"
11 #include "irrigation_timer.h"
12
13 //=====[Declaration of private defines]========================================
14
```

```
15  static nonBlockingDelay_t irrigationSystemDelay;
16
17  //=====[Implementations of public functions]===================================
18
19  void irrigationSystemInit()
20  {
21      tickInit();
22      buttonsInit();
23      moistureSensorInit();
24      relayInit();
25      irrigationControlInit();
26      irrigationTimerInit();
27      userInterfaceInit();
28      nonBlockingDelayInit( &irrigationSystemDelay, SYSTEM_TIME_INCREMENT_MS );
29  }
30
31  void irrigationSystemUpdate()
32  {
33      if( nonBlockingDelayRead(&irrigationSystemDelay) ) {
34          buttonsUpdate();
35          moistureSensorUpdate();
36          relayUpdate();
37          irrigationControlUpdate();
38          irrigationTimerUpdate();
39          userInterfaceUpdate();
40      }
41  }
```

Code 12.17 New implementation of irrigation_system.cpp.

In Code 12.18, the implementation of *irrigation_control.h* is shown. On line 8, TO_SECONDS is defined as 10. This #define will be used to convert from a number of counts of 100 milliseconds to seconds. On line 9, TO_HOURS is defined as 36000. This #define will be used to convert from a number of counts of 100 milliseconds to hours.

On line 13, the public data type *irrigationState_t* is declared. As can be seen on lines 14 to 18, it can have five possible values. On line 21, the public data type *irrigationControlStatus_t* is declared. Its first member is *irrigationState*, of type *irrigationState_t*. The other two members are the Boolean variables, *waitedTimeMustBeReset* and *irrigatedTimeMustBeReset*. On lines 29 to 31, the prototypes of the public functions *irrigationControlInit()*, *irrigationControlUpdate()*, and *irrigationControlRead()* are declared.

```
1   //=====[#include guards - begin]===============================================
2
3   #ifndef _IRRIGATION_CONTROL_H_
4   #define _IRRIGATION_CONTROL_H_
5
6   //=====[Declaration of public defines]=========================================
7
8   #define TO_SECONDS   10
9   #define TO_HOURS     36000
10
11  //=====[Declaration of public data types]======================================
12
13  typedef enum {
14      INITIAL_MODE_ASSESSMENT,
```

```
15      CONTINUOUS_MODE_IRRIGATING,
16      PROGRAMMED_MODE_WAITING_TO_IRRIGATE,
17      PROGRAMMED_MODE_IRRIGATION_SKIPPED,
18      PROGRAMMED_MODE_IRRIGATING
19  } irrigationState_t;
20
21  typedef struct irrigationControlStatus {
22      irrigationState_t irrigationState;
23      bool waitedTimeMustBeReset;
24      bool irrigatedTimeMustBeReset;
25  } irrigationControlStatus_t;
26
27  //=====[Declarations (prototypes) of public functions]=========================
28
29  void irrigationControlInit();
30  void irrigationControlUpdate();
31  irrigationControlStatus_t irrigationControlRead();
32
33  //=====[#include guards - end]=================================================
34
35  #endif // _IRRIGATION_CONTROL_H_
```

Code 12.18 Implementation of irrigation_control.h.

In Code 12.19 and Code 12.20, the implementation of *irrigation_control.cpp* is shown. The libraries are included on lines 3 to 10 of Code 12.19. A private global variable named *irrigationControlStatus* is declared on line 14. The function *irrigationControlInit()*, declared on line 18, is used to initialize the members of *irrigationControlStatus*. The function *irrigationControlUpdate()*, shown from lines 25 to 65 of Code 12.19 and 1 to 40 of Code 12.20, implements the FSM discussed in Example 12.4. It is, therefore, not discussed here. Finally, in Code 12.20, the function *irrigationControlRead()* is implemented on lines 42 to 45 of Code 12.20.

NOTE: FSM transitions corresponding to *waitedTime* < *howOften*, *irrigatedTime* < *howLong*, *changeMode* == false (see Figure 12.7) are to the same state, so there is no need to include code to implement them. Lines 8 to 10 of Code 12.20 are included only to improve code clarity but can be replaced by *else irrigationControlStatus. irrigationState* = PROGRAMMED_MODE_IRRIGATION_SKIPPED.

```
1   //=====[Libraries]=============================================================
2
3   #include "mbed.h"
4   #include "arm_book_lib.h"
5
6   #include "irrigation_control.h"
7
8   #include "buttons.h"
9   #include "irrigation_timer.h"
10  #include "moisture_sensor.h"
11
12  //=====[Declaration and initialization of private global variables]============
13
14  static irrigationControlStatus_t irrigationControlStatus;
15
```

```
16  //=====[Implementations of public functions]=================================
17
18  void irrigationControlInit()
19  {
20      irrigationControlStatus.irrigationState = INITIAL_MODE_ASSESSMENT;
21      irrigationControlStatus.waitedTimeMustBeReset = true;
22      irrigationControlStatus.irrigatedTimeMustBeReset = true;
23  }
24
25  void irrigationControlUpdate()
26  {
27      buttonsStatus_t buttonsStatusLocalCopy;
28      irrigationTimer_t irrigationTimerLocalCopy;
29      float h169AveragedValueLocalCopy;
30
31      buttonsStatusLocalCopy = buttonsRead();
32      irrigationTimerLocalCopy = irrigationTimerRead();
33      h169AveragedValueLocalCopy = moistureSensorRead();
34
35      switch( irrigationControlStatus.irrigationState ) {
36
37          case INITIAL_MODE_ASSESSMENT:
38
39              if( buttonsStatusLocalCopy.changeMode ) {
40                 irrigationControlStatus.irrigationState = CONTINUOUS_MODE_IRRIGATING;
41              } else {
42                     irrigationControlStatus.irrigationState =
43                                                 PROGRAMMED_MODE_WAITING_TO_IRRIGATE;
44                  irrigationControlStatus.waitedTimeMustBeReset = true;
45              }
46              break;
47
48          case CONTINUOUS_MODE_IRRIGATING:
49
50              if( buttonsStatusLocalCopy.changeMode ) {
51                  irrigationControlStatus.irrigationState =
52                                                 PROGRAMMED_MODE_WAITING_TO_IRRIGATE;
53                  irrigationControlStatus.waitedTimeMustBeReset = true;
54              }
55              break;
56
57          case PROGRAMMED_MODE_WAITING_TO_IRRIGATE:
58
59              irrigationControlStatus.waitedTimeMustBeReset = false;
60              if( buttonsStatusLocalCopy.changeMode ) {
61                  irrigationControlStatus.irrigationState =
62                                                 CONTINUOUS_MODE_IRRIGATING;
63              }
64              else if( irrigationTimerLocalCopy.waitedTime >= (
65                       buttonsStatusLocalCopy.howOften * TO_HOURS) ) {
```

Code 12.19 Implementation of irrigation_control.cpp (Part 1/2).

```
                if( ( buttonsStatusLocalCopy.howLong != 0 ) &&
                    ( (int) (100*hl69AveragedValueLocalCopy) <
                       buttonsStatusLocalCopy.moisture ) ) {
                irrigationControlStatus.irrigationState =
                                            PROGRAMMED_MODE_IRRIGATING;
                irrigationControlStatus.irrigatedTimeMustBeReset = true;
                }
                else if ( ( buttonsStatusLocalCopy.howLong == 0 ) ||
                          ( (int) (100*hl69AveragedValueLocalCopy) >=
                            buttonsStatusLocalCopy.moisture ) ) {
                irrigationControlStatus.irrigationState =
                                            PROGRAMMED_MODE_IRRIGATION_SKIPPED;
                }
            }

        case PROGRAMMED_MODE_IRRIGATION_SKIPPED:

            irrigationControlStatus.waitedTimeMustBeReset = true;
           irrigationControlStatus.irrigationState =
                                            PROGRAMMED_MODE_WAITING_TO_IRRIGATE;

            break;

        case PROGRAMMED_MODE_IRRIGATING:

            irrigationControlStatus.waitedTimeMustBeReset = false;

            if( irrigationTimerLocalCopy.irrigatedTime >= (
                buttonsStatusLocalCopy.howLong * TO_SECONDS) ) {
                irrigationControlStatus.irrigationState =
                                            PROGRAMMED_MODE_WAITING_TO_IRRIGATE;
                irrigationControlStatus.waitedTimeMustBeReset = true;
            }
            break;

        default:
            irrigationControlInit();
            break;
    }
}

irrigationControlStatus_t irrigationControlRead()
{
    return irrigationControlStatus;
}
```

Code 12.20 Implementation of irrigation_control.cpp (Part 2/2).

In Code 12.21, the implementation of *irrigation_timer.h* is shown. On line 8, the defined type *irrigationTimer_t* is declared. It has two members, *waitedTime* and *irrigatedTime*, used to account for the time elapsed while waiting to irrigate and the time elapsed while irrigating. On lines 15 to 17, the public functions are declared.

In Code 12.22, the implementation of *irrigation_timer.cpp* is shown. Libraries are included on lines 3 to 8. On line 12, the defined type variable *irrigationTimer* is declared. On lines 16 to 20, *irrigationTimerInit()* is implemented. It can be seen that *irrigationTimer.waitedTime* and *irrigationTimer.irrigatedTime* are both set to 0. On lines 22 to 45, *irrigationTimerUpdate()* is implemented. The variable *irrigationControlStatusLocalCopy* is declared on line 24, and it is assigned the return value of

irrigationControlRead() on line 26. Line 28 assesses whether *irrigationTimer.waitedTime* must be reset by evaluating *irrigationControlStatusLocalCopy.waitedTimeMustBeReset*. Line 32 assesses whether *irrigationTimer.irrigatedTime* must be reset.

Line 36 assesses whether *irrigationTimer.waitedTime* must be incremented, which is true if *irrigationControlStatusLocalCopy.irrigationState* is equal to PROGRAMMED_MODE_WAITING_TO_IRRIGATE. Line 41 assesses whether *irrigationTimer.irrigatedTime* must be incremented, which is true if *irrigationControlStatusLocalCopy.irrigationState* is equal to PROGRAMMED_MODE_IRRIGATING. Finally, on lines 47 to 50, *irrigationTimerRead()* is implemented.

```
1  //=====[#include guards - begin]===============================================
2
3  #ifndef _IRRIGATION_TIMER_H_
4  #define _IRRIGATION_TIMER_H_
5
6  //=====[Declaration of public data types]======================================
7
8  typedef struct irrigationTimer {
9      int waitedTime;
10     int irrigatedTime;
11 } irrigationTimer_t;
12
13 //=====[Declarations (prototypes) of public functions]=========================
14
15 void irrigationTimerInit();
16 void irrigationTimerUpdate();
17 irrigationTimer_t irrigationTimerRead();
18
19 //=====[#include guards - end]=================================================
20
21 #endif // _IRRIGATION_TIMER_H_
```

Code 12.21 Implementation of irrigation_timer.h.

```
1  //=====[Libraries]=============================================================
2
3  #include "mbed.h"
4  #include "arm_book_lib.h"
5
6  #include "irrigation_timer.h"
7
8  #include "buttons.h"
9
10 //=====[Declaration and initialization of private global variables]============
11
12 static irrigationTimer_t irrigationTimer;
13
14 //=====[Implementations of public functions]===================================
15
16 void irrigationTimerInit()
17 {
18     irrigationTimer.waitedTime = 0;
19     irrigationTimer.irrigatedTime = 0;
20 }
21
22 void irrigationTimerUpdate()
23 {
24     irrigationControlStatus_t irrigationControlStatusLocalCopy;
```

```
25
26      irrigationControlStatusLocalCopy = irrigationControlRead();
27
28      if ( irrigationControlStatusLocalCopy.waitedTimeMustBeReset ) {
29          irrigationTimer.waitedTime = 0;
30      }
31
32      if ( irrigationControlStatusLocalCopy.irrigatedTimeMustBeReset ) {
33          irrigationTimer.irrigatedTime = 0;
34      }
35
36      if ( irrigationControlStatusLocalCopy.irrigationState ==
37          PROGRAMMED_MODE_WAITING_TO_IRRIGATE ) {
38          irrigationTimer.waitedTime++;
39      }
40
41      if ( irrigationControlStatusLocalCopy.irrigationState ==
42          PROGRAMMED_MODE_IRRIGATING ) {
43          irrigationTimer.irrigatedTime++;
44      }
45  }
46
47  irrigationTimer_t irrigationTimerRead()
48  {
49      return irrigationTimer;
50  }
```

Code 12.22 Implementation of irrigation_timer.cpp.

In Code 12.23 and Code 12.24, the new implementation of *user_interface.cpp* is shown. In Code 12.23, libraries are included on lines 3 to 11, and line 21 is modified to only write "Mode:". On line 9 of Code 12.24, the display position after "Mode:" is written (i.e., "5,0"). Then, a switch statement is used on *irrigationControlStatusLocalCopy.irrigationState* to write the corresponding text on the display.

```
1   //=====[Libraries]=============================================================
2
3   #include "mbed.h"
4   #include "arm_book_lib.h"
5
6   #include "user_interface.h"
7
8   #include "display.h"
9   #include "buttons.h"
10  #include "moisture_sensor.h"
11  #include "irrigation_control.h"
12
13  //=====[Implementations of public functions]===================================
14
15  void userInterfaceInit()
16  {
17      displayInit( DISPLAY_TYPE_LCD_HD44780,DISPLAY_CONNECTION_GPIO_4BITS );
18
19      displayClear();
20      displayCharPositionWrite( 0, 0 );
21      displayStringWrite( "Mode:" );
22      displayCharPositionWrite( 0, 1 );
23      displayStringWrite( "HowOften:  hours" );
24      displayCharPositionWrite( 0, 2 );
```

```
25      displayStringWrite( "HowLong:  seconds" );
26      displayCharPositionWrite( 0, 3 );
27      displayStringWrite( "MinMois:  %-Curr:  %" );
28  }
29
30  void userInterfaceUpdate()
31  {
32      char number[3];
33
34      buttonsStatus_t buttonsStatusLocalCopy;
35      float hl69AveragedValueLocalCopy;
36      irrigationControlStatus_t irrigationControlStatusLocalCopy;
37
38      buttonsStatusLocalCopy = buttonsRead();
39      hl69AveragedValueLocalCopy = moistureSensorRead();
40      irrigationControlStatusLocalCopy = irrigationControlRead();
41
42      displayCharPositionWrite( 9, 1 );
43      sprintf( number, "%02d", buttonsStatusLocalCopy.howOften );
44      displayStringWrite( number );
45
46      displayCharPositionWrite( 8, 2 );
47      sprintf( number, "%02d", buttonsStatusLocalCopy.howLong );
48      displayStringWrite( number );
49
```

Code 12.23 New implementation of user_interface.cpp (Part 1/2).

```
 1      displayCharPositionWrite( 8, 3 );
 2      sprintf( number, "%02d", buttonsStatusLocalCopy.moisture );
 3      displayStringWrite( number );
 4
 5      displayCharPositionWrite( 17, 3 );
 6      sprintf( number, "%2.0f", 100*hl69AveragedValueLocalCopy );
 7      displayStringWrite( number );
 8
 9      displayCharPositionWrite( 5, 0 );
10
11      switch( irrigationControlStatusLocalCopy.irrigationState ) {
12
13          case INITIAL_MODE_ASSESSMENT:
14              displayStringWrite( "Initializing..." );
15              break;
16
17          case CONTINUOUS_MODE_IRRIGATING:
18              displayStringWrite( "Continuous-ON  " );
19              break;
20
21          case PROGRAMMED_MODE_WAITING_TO_IRRIGATE:
22              displayStringWrite( "Programmed-Wait" );
23              break;
24
25          case PROGRAMMED_MODE_IRRIGATION_SKIPPED:
26              displayStringWrite( "Programmed-Skip" );
27              break;
28
29          case PROGRAMMED_MODE_IRRIGATING:
30              displayStringWrite( "Programmed-ON  " );
31              break;
32
33          default:
34              displayStringWrite( "Non-supported  " );
35              break;
36      }
37
38  void userInterfaceRead()
39  {
40  }
```

Code 12.24 New implementation of user_interface.cpp (Part 2/2).

In Code 12.25, the new implementation of *relay.cpp* is shown. Libraries are included on lines 3 to 8. Note that *buttons.h* is not included anymore. The private DigitalInOut object *relayControlPin* is declared on line 12 and assigned to pin PF_2. The function *relayInit()* is declared on line 16. This function turns off the relay by configuring *relayControlPin* as an input, so no energy is supplied to the relay.

On lines 22 to 56, the implementation of *relayUpdate()* is shown. On line 24, *irrigationControlStatusLocalCopy* is declared and assigned the return value of *irrigationControlRead()* on line 26. On line 28, there is a switch over *irrigationControlStatusLocalCopy.irrigationState*. Depending on the value of *irrigationControlStatusLocalCopy.irrigationState*, the relay is turned on or off. For example, in the irrigation state CONTINUOUS_MODE_IRRIGATING, the relay should be turned on, and, therefore, *relayControlPin* is configured as output and LOW is assigned to the pin (remember that the relay is turned on by assigning 0 V to the pin PF_2).

Finally, the function *relayRead()* is shown on lines 57 to 61.

```
1  //=====[Libraries]=============================================================
2
3  #include "mbed.h"
4  #include "arm_book_lib.h"
5
6  #include "relay.h"
7
8  #include "irrigation_control.h"
9
10 //=====[Declaration and initialization of public global objects]===============
11
12 static DigitalInOut relayControlPin(PF_2);
13
14 //=====[Implementations of public functions]===================================
15
16 void relayInit()
17 {
18     relayControlPin.mode(OpenDrain);
19     relayControlPin.input();
20 }
21
22 void relayUpdate()
23 {
24     irrigationControlStatus_t irrigationControlStatusLocalCopy;
25
26     irrigationControlStatusLocalCopy = irrigationControlRead();
27
28     switch( irrigationControlStatusLocalCopy.irrigationState ) {
29
30         case INITIAL_MODE_ASSESSMENT:
31             relayControlPin.input();
32             break;
33
34         case CONTINUOUS_MODE_IRRIGATING:
35             relayControlPin.output();
36             relayControlPin = LOW;
37             break;
38
39         case PROGRAMMED_MODE_WAITING_TO_IRRIGATE:
40             relayControlPin.input();
```

```
41              break;
42
43          case PROGRAMMED_MODE_IRRIGATION_SKIPPED:
44              relayControlPin.input();
45              break;
46
47          case PROGRAMMED_MODE_IRRIGATING:
48              relayControlPin.output();
49              relayControlPin = LOW;
50              break;
51
52          default:
53              relayControlPin.input();
54              break;
55      }
56  }
57
58  void relayRead()
59  {
60  }
```

Code 12.25 New implementation of relay.cpp.

Proposed Exercise

1. Implement the behavior for the project that was selected in the "Proposed Exercise" of Example 12.1. Write all the corresponding *.h* and *.cpp* files.

Answer to the Exercise

1. The implementation of the behavior might be a difficult task. For that reason, some tips are presented below.

TIP: It is strongly recommended to start by implementing a reduced set of the system functionality. In this way, the code is easier to revise and test. For example, in Code 12.26, it is shown how a first version of *irrigationControlUpdate()* could be written to include only three of the states.

```
 1  void irrigationControlUpdate()
 2  {
 3      buttonsStatus_t buttonsStatusLocalCopy;
 4
 5      buttonsStatusLocalCopy = buttonsRead();
 6
 7      switch( irrigationControlStatus.irrigationState ) {
 8
 9          case INITIAL_MODE_ASSESSMENT:
10
11              if( buttonsStatusLocalCopy.changeMode ) {
12                 irrigationControlStatus.irrigationState = CONTINUOUS_MODE_IRRIGATING;
13              } else {
14                     irrigationControlStatus.irrigationState =
15                                              PROGRAMMED_MODE_WAITING_TO_IRRIGATE;
16                  }
```

```
17              break;
18
19          case CONTINUOUS_MODE_IRRIGATING:
20
21              if( buttonsStatusLocalCopy.changeMode ) {
22                  irrigationControlStatus.irrigationState =
23                                              PROGRAMMED_MODE_WAITING_TO_IRRIGATE;
24              }
25              break;
26
27          case PROGRAMMED_MODE_WAITING_TO_IRRIGATE:
28              if( buttonsStatusLocalCopy.changeMode ) {
29                  irrigationControlStatus.irrigationState =
30                                              CONTINUOUS_MODE_IRRIGATING;
31                }
32              break;
33
34          default:
35              irrigationControlInit();
36              break;
37      }
38  }
```

Code 12.26 Simplified implementation of irrigation_control.cpp.

TIP: It is also recommended to consider showing some additional information on the user interface to have a better understanding of what is going on. For example, lines 59 to 65 of Code 12.27 are used to print on the display the values of *waitedTime* and *irrigatedTime*. By means of these lines, as well as the values of TO_SECONDS and TO_HOUR shown in Table 12.34, the behavior of the system can be tested in a shorter amount of time by having more information on hand.

```
1   void userInterfaceUpdate()
2   {
3       char number[3];
4
5       buttonsStatus_t buttonsStatusLocalCopy;
6       float hl69AveragedValueLocalCopy;
7       irrigationControlStatus_t irrigationControlStatusLocalCopy;
8       irrigationTimer_t irrigationTimerLocalCopy;
9
10      buttonsStatusLocalCopy = buttonsRead();
11      hl69AveragedValueLocalCopy = moistureSensorRead();
12      irrigationControlStatusLocalCopy = irrigationControlRead();
13      irrigationTimerLocalCopy = irrigationTimerRead();
14
15      displayCharPositionWrite( 9, 1 );
16      sprintf( number, "%02d", buttonsStatusLocalCopy.howOften );
17      displayStringWrite( number );
18
19      displayCharPositionWrite( 8, 2 );
20      sprintf( number, "%02d", buttonsStatusLocalCopy.howLong );
21      displayStringWrite( number );
22
23      displayCharPositionWrite( 8, 3 );
24      sprintf( number, "%02d", buttonsStatusLocalCopy.moisture );
25      displayStringWrite( number );
```

```
26
27      displayCharPositionWrite( 17, 3 );
28      sprintf( number, "%2.0f", 100*hl69AveragedValueLocalCopy );
29      displayStringWrite( number );
30
31      displayCharPositionWrite( 5, 0 );
32      switch( irrigationControlStatusLocalCopy.irrigationState ) {
33
34          case INITIAL_MODE_ASSESSMENT:
35              displayStringWrite( "Initializing..." );
36              break;
37
38          case CONTINUOUS_MODE_IRRIGATING:
39              displayStringWrite( "Continuous-ON  " );
40              break;
41
42          case PROGRAMMED_MODE_WAITING_TO_IRRIGATE:
43              displayStringWrite( "Programmed-Wait" );
44              break;
45
46          case PROGRAMMED_MODE_IRRIGATION_SKIPPED:
47              displayStringWrite( "Programmed-Skip" );
48              break;
49
50          case PROGRAMMED_MODE_IRRIGATING:
51              displayStringWrite( "Programmed-ON  " );
52              break;
53
54          default:
55              displayStringWrite( "Non-supported  " );
56              break;
57      }
58
59      displayCharPositionWrite( 18, 1 );
60      sprintf( number, "%02d", irrigationTimerLocalCopy.waitedTime );
61      displayStringWrite( number );
62
63      displayCharPositionWrite( 18, 2 );
64      sprintf( number, "%02d", irrigationTimerLocalCopy.irrigatedTime );
65      displayStringWrite( number );
66  }
```

Code 12.27 Alternative version of the implementation of user_interface.cpp, where more information is shown.

Table 12.34 Sections in which lines were modified in irrigation_control.h.

Section	Lines that were added
Declaration of public defines	`#define TO_SECONDS 1` `#define TO_HOURS 1`

WARNING: The UART connection with the PC over USB can be used to print the values of *waitedTime* and *irrigatedTime* on the serial terminal. In this case, it should be remembered that there is a delay for each character that is sent to the PC (recall the Under the Hood section of Chapter 2), and this may alter the system behavior.

Example 12.9: Check the System Behavior

Objective

Check the behavior of the system against the requirements and use cases defined in step 2.

Summary of the Expected Outcome

The results of this step are expected to be:

- a table listing all the requirements, indicating whether they were accomplished or not, and
- a table listing all the use cases, indicating whether the system behaves as expected or not.

Discussion of How to Implement this Step

In step 2 (Example 12.2), the requirements were established, as well as the use cases. In this step, the accomplishment of each requirement and use case will be analyzed. In the case of non-compliance, it will be explained why it was not possible to accomplish the requirement and what the proposed solution is.

Implementation of this Proposed Step

In Table 12.35, the accomplishment of the requirements is assessed. It can be seen that out of 24 requirements, only 2 were not accomplished (Req. 5.1 and 6.1). In the case of Req. 5.1, the moisture sensor that was used is not, and cannot be, calibrated. Perhaps in a future version of the system the sensor can be changed or calibrated. Regarding Req. 6.1, it was noticed that the available solenoid valves operate with 12 V, and for the sake of simplicity in this version it was decided not to use an extra circuit (e.g., a step-up DC/DC converter) to obtain 12 V out of two AA batteries.

Table 12.35 Accomplishment of the requirements defined for the home irrigation system.

Req. ID	Description	Accomplished?
1.1	The system will have one water-in port based on a ½-inch connector.	✓
1.2	The system will control one irrigation circuit by means of a solenoid valve.	✓
2.1	The system will have a continuous mode in which a button will enable the water flow.	✓
2.2	The system will have a programmed irrigation mode based on a set of configurations:	✓
2.2.1	Irrigation will be enabled only if moisture is below the “Minimum moisture level” value.	✓
2.2.2	Irrigation will be enabled every H hours, with H being the “How often” configuration.	✓
2.2.3	Irrigation will be enabled for S seconds, with S being the “How long” configuration.	✓
2.2.4	Irrigation will be skipped if “How long” is configured to 0 (zero).	✓
3.1	The system configuration will be done by means of a set of buttons:	✓
3.1.1	The “Mode” button will change between “Programmed irrigation” and “Continuous irrigation”.	✓
3.1.2	The “How often” button will increase the time between irrigations in programmed mode by one hour.	✓
3.1.3	The “How long” button will increase the irrigation time in programmed mode by ten seconds.	✓

Req. ID	Description	Accomplished?
3.1.4	The "Moisture" button will increase the "Minimum moisture level" configuration by 5%.	✓
3.1.5	The maximum values are: "How often": 24 h; "How long": 90 s; "Moisture": 95%.	✓
3.1.6	"How long" and "Moisture" will be set to 0 (zero) if they reach the maximum and the button is pressed.	✓
3.1.7	"How often" will be set to 1 if it reaches its maximum value and the button is pressed.	✓
4.1	The system will have an LCD display:	✓
4.1.1	The LCD display will show the current operation mode: Continuous or Programmed.	✓
4.1.2	The LCD display will show the value of "How often", "How long", and "Moisture".	✓
5.1	The system will measure soil moisture with an accuracy better than 5%.	×
6.1	The system will be powered using two AA batteries.	×
7.1	The system should be finished one week after starting (this includes buying the parts).	✓
8.1	The components for the prototype should cost less than 60 USD.	✓
9.1	The prototype should be accompanied with a list of parts, a connection diagram, the code repository, a table indicating the accomplishment of requirements, and use cases.	✓

NOTE: Req. 7.1 is considered accomplished based on an estimation of the time that it will take the reader to complete this project, considering the skills acquired through this book and six hours/day of work.

In Table 12.36, the set of use cases that were established in Example 12.2 are shown, as is an assessment of whether they have been accomplished. It can be seen that the three use cases were fully accomplished.

Table 12.36 Accomplishment of the use cases defined for the home irrigation system.

Use case	Title	Accomplished?
#1	The user wants to irrigate plants immediately for a couple of minutes.	✓
#2	The user wants to program irrigation to take place for ten seconds every six hours.	✓
#3	The user wants the plants not to be irrigated.	✓

Considering that 92% of the requirements and 100% of the use cases were accomplished, it can be considered that the project was successfully developed.

Lastly, this helps to introduce two important concepts, which are frequently confused with each other: *verification* and *validation* [3].

DEFINITION: *Verification* asks the question, "Are we building the product right?" That is, does the software conform to its specifications? (As a house conforms to its blueprints.)

DEFINITION: *Validation* asks the question, "Are we building the right product?" That is, does the software do what the user really requires? (As a house conforms to what the owner wants.)

After these definitions, and given the accomplishment results shown above, it can be stated that:

- It was verified that the system that was developed accomplished almost all of its specifications.
- Given that users were not asked about their needs, at this point validation cannot be assessed.

It can be concluded that it is of capital importance to involve the users at an early stage of the system development process in order to be able to adjust the system specifications to their actual needs. Otherwise, a product can be built that works as expected but is not useful for the users.

Proposed Exercise

1. Analyze the accomplishment of the requirements and use cases of the project that was selected in the "Proposed Exercise" of Example 12.1. Document them by means of tables, as has been shown in this example.

Answer to the Exercise

1. Table 12.35 and Table 12.36 can be used as templates to list all the requirements and use cases and the corresponding assessment of their accomplishment.

Example 12.10: Develop the Documentation of the System

Objective

Provide a set of documents that summarize the final outcome of the project.

Summary of the Expected Outcome

The results of this step are expected to be:

- a set of tables, drawings, and rationales summarizing the results of the project, and
- a list of proposed steps that could be followed in order to continue and improve the project.

Discussion of How to Implement this Step

Throughout the examples, many tables, drawings, and rationales about the project were shown. In this final step, all those documents can be presented as a final summary of the project. Additionally, based on the experience obtained during the project implementation, proposals can be made about how to continue and improve the project.

Implementation of this Proposed Step

In Table 12.37, a set of elements that summarize the most important information about the home irrigation system design and implementation are presented. It can be seen that by analyzing this information, a developer can understand most of the project details. A user manual and complementary documents can be added to this list, depending on the specific characteristics of the project.

Table 12.37 Elements that summarize the most important information about the home irrigation system.

Element	Reference
Rationale explaining why the project was selected	Example 12.1
Requirements of the project	Table 12.6
Use cases of the project	Table 12.7
Diagram of the hardware modules of the system	Figure 12.2
Connection diagram of all the hardware elements	Figure 12.3
Bill of materials	Table 12.18
Diagram of the software design	Figure 12.4, Figure 12.5
Definition of the software modules (public functions, variables, etc.)	Table 12.19 to Table 12.32
Diagram of the proposed finite-state machine	Figure 12.7
Software implementation	[2]
Assessment of the accomplishment of the requirements	Table 12.35
Assessment of the accomplishment of the use cases	Table 12.36
Proposal for next steps	Example 12.10
Final conclusions	Example 12.10

Based on the experience obtained during the project design and implementation, the following next steps can be addressed:

- A step-up module might be included in the design in order to provide the 5 V supply for the NUCLEO board and the 12 V supply for the solenoid using two AA batteries.
- A calibration process can be implemented to determine the correspondence between the output signal of the HL-69 sensor and the moisture level.

Moreover, in order to achieve a commercial version of the design, the following items can be considered:

- The development board might be replaced by a bespoke design on a printed circuit board (PCB), which should reduce the cost and size of the system.

- A case can be designed for the irrigation system in order to make it portable and usable for final users.

- The reliability of the system can be tested in order to determine if more improvements are required.

As a final conclusion for this project, it can be seen that the software design proved to be appropriate for implementing a project with a certain complexity. Therefore, the approach can be reused in future projects, such as the mobile robot that avoids obstacles or the flying drone equipped with a camera, which were mentioned in Example 12.1.

Proposed Exercise

1. Develop a list of the relevant documentation regarding the project which was selected in the "Proposed Exercise" of Example 12.1. Document the project by means of tables, as has been shown in this example. Recommend a set of future steps for the project, and finally develop a brief conclusion.

Answer to the Exercise

1. Table 12.37 can be used as a template to list all of the relevant documentation. The next steps and the final conclusion can be similar to the ones presented in this example.

12.3 Final Words

12.3.1 The Projects to Come

Throughout this book, many concepts have been introduced. At this point, the reader is capable of implementing a broad set of embedded systems. Moreover, the reader now has many keys that will open the doors to "projects to come." Those projects will include some technologies and techniques that were introduced and discussed in this book, and probably some other elements that are beyond the scope of this book. In any case, the reader now has a solid basis into which new knowledge can be incorporated.

For example, the HC-SR04 ultrasonic module shown in Figure 12.12 is very popular. It uses the principle of sonar (sound navigation ranging) to determine the distance to an object in the same way that bats do. It has four pins: VCC, Trig, Echo, and GND. The Trig pin is used to trigger a high-frequency sound. When the signal is reflected, the echo pin changes its state. The time between the transmission and reception of the signal allows us to calculate the distance to an object, which is useful, for example, in parking systems or in autonomous vehicles. Note that the knowledge to implement the appropriate software module to use this sensor was introduced within this book (i.e., look at the Tip on Example 10.4).

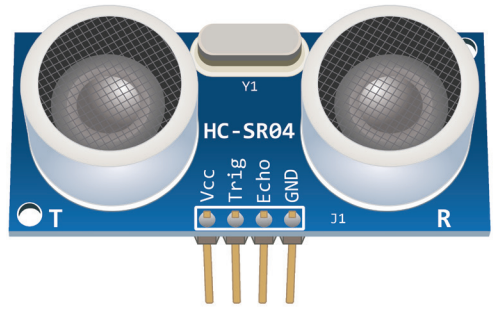

Figure 12.12 Image of the HC-SR04 ultrasonic module.

A microphone module, as shown in Figure 12.13, can be easily used by applying the concepts introduced in this book. The module has a digital output pin (DO) and analog output pin (AO). The digital output sends a high signal when the sound intensity reaches a certain threshold, which is adjusted using the potentiometer on the sensor. The analog output can be sampled and stored in an array using an appropriate sampling rate, in order to be reproduced later, for example in a public address system. The procedure is similar to that applied to the temperature sensor.

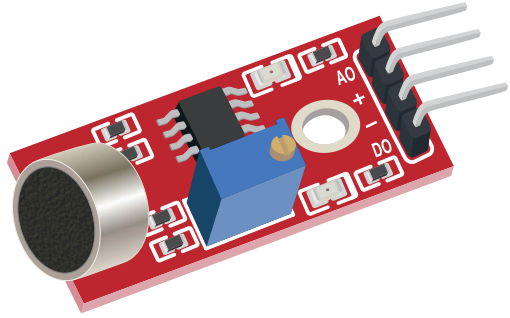

Figure 12.13 Image of a microphone module.

In Figure 12.14, a digital barometric pressure module is shown, used to measure the absolute pressure of the environment. By converting the pressure measures into altitude, the height of a drone can be estimated. The sensor also measures temperature and humidity and has an I2C bus interface, such as the one introduced in Chapter 6.

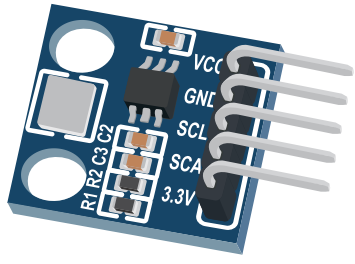

Figure 12.14 Image of a digital barometric pressure module.

There are many other sensor modules available, as well as many actuators that the reader is ready to use. For example, the micro servo motor shown in Figure 12.15 has three pins: GND, VCC, and Signal. The angle of the motor axis angle is controlled in the range 0 to 180° by the duty cycle of the signal delivered at the Signal pin. The reader is encouraged to explore actuators that allow tiny movements, such as stepper motors.

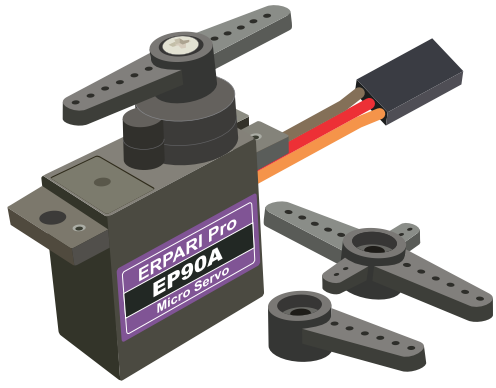

Figure 12.15 Image of a micro servo motor.

GPS (global positioning system) modules, such as the one shown in Figure 12.16, are also very popular. They are commanded using AT commands, in a very similar way to the ESP-01 module introduced in Chapter 11.

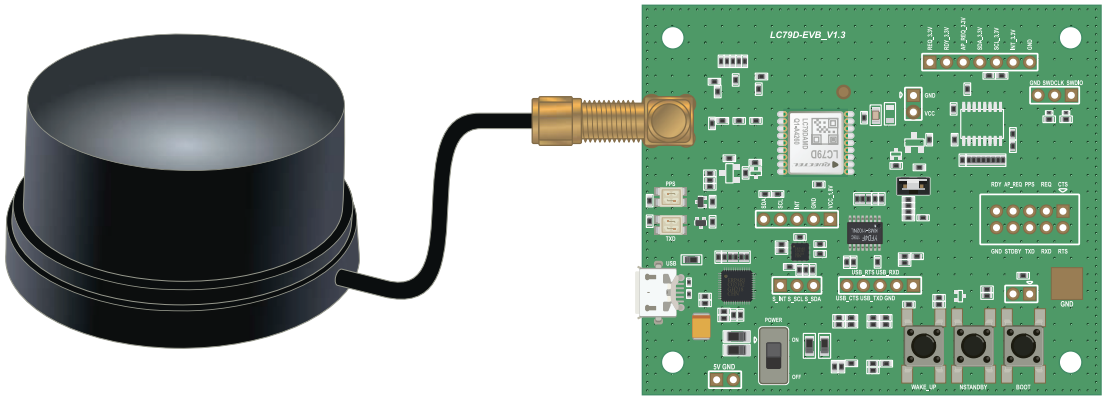

Figure 12.16 Image of a GPS module. Note the GPS antenna on the left.

Finally, in Figure 12.17, an NB-IoT (Narrow Band Internet of Things) cellular module is shown. These modules are also controlled using AT commands and can be used to implement communications based on 4G, 5G, LoRa, or SigFox, for example.

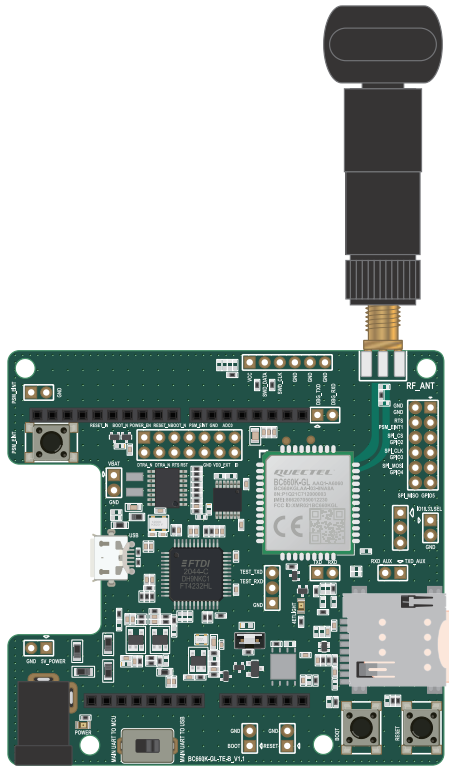

Figure 12.17 Image of an NB-IoT cellular module. Note the SIM card slot on the right.

Given the vast range of modules that are available, the aim of this section is just to give an overview of how most of them, maybe even all of them, can be used by applying the techniques and concepts introduced in this book.

Enjoy the projects to come!

TIP: Never forget that good engineering is based on following processes, keeping things as simple as possible, dividing problems into small pieces, and solving trade-off situations.

Proposed Exercise

1. How can a micro servo motor be controlled using the techniques introduced in this book?

Answer to the Exercise

1. As explained above, the motor axis angle is controlled by the duty cycle of the signal delivered at the Signal pin. Thus, the PWM technique that was introduced in Chapter 8 can be used.

References

[1] "NUCLEO-F429ZI | Mbed". Accessed July 9, 2021. https://os.mbed.com/platforms/ST-Nucleo-F429ZI/#zio-and-arduino-compatible-headers

[2] "GitHub - armBookCodeExamples/Directory". Accessed July 9, 2021. https://github.com/armBookCodeExamples/Directory

[3] Pham, H. (1999). Software Reliability. John Wiley & Sons, Inc. p. 567. ISBN 9813083840.

Glossary of Abbreviations

4G	Fourth Generation
A	Anode
AC	Alternating Current
ACK	Acknowledge
ACSE	Asociación Civil para la Investigación, Promoción y Desarrollo de Sistemas Electrónicos Embebidos, Civil Association for Research, Promotion and Development of Embedded Electronic Systems
ADC	Analog to Digital Converter
ALT	Alternative
AOUT, AO	Analog Output
API	Application Programming Interface
ASCII	American Standard Code For Information Interchange
AT	Attention
BLE	Bluetooth Low Energy
BRK	Break
CADIEEL	Cámara Argentina de Industrias Electrónicas, Electromecánicas y Luminotécnicas, Argentine Chamber of Electronic, Electromechanical and Lighting Industries
CGRAM	Custom Generated Random-Access Memory
CIAA	Computadora Industrial Abierta Argentina, Argentine Open Industrial Computer
CO_2	Carbon Dioxide
COM	Communication